



Fakultät Elektronik und Informatik

Studiengang Informatik/Security

Stoneforge RL

Navigation in einer eigenentwickelten prozedural generierten 2D Welt
mittels Reinforcement Learning

Projektarbeit

Vorgelegt von: Florian Merlau, 81775
Laurin Rößler

Betreuer: Prof. Lecon

Abgabedatum: 14. 08. 2026

Aalen, 2026

Eidesstattliche Erklärung

Hiermit wird an Eides statt erklärt, dass die vorliegende Projektarbeit mit dem Titel

*»Stoneforge RL — Navigation in einer prozedural generierten,
partiell beobachtbaren 2D-Welt mittels Reinforcement Learning«*

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt wurden. Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder unveröffentlichten Quellen entnommen sind, werden als solche kenntlich gemacht. Die Arbeit wurde bisher in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegt.

Da die Arbeit von zwei Autoren gemeinsam erstellt wurde, wird zusätzlich erklärt, dass Kapitel 3.1, 4.1 sowie Abschnitt 5.1 von Laurin ... und Kapitel 3.2, 4.2 sowie Abschnitt 5.2 von Florian ... verfasst wurden; die gemeinsamen Kapitel 1, 2 und 6 entstanden in gemeinsamer Autorenschaft.

Aalen, den 14.08.2026

Aalen, den 14.08.2026

Florian ...

Laurin ...

Kurzfassung

Die vorliegende Arbeit untersucht, welche Kombination aus Policy-Architektur und Trainingsverfahren die Navigationsaufgabe in einer prozedural generierten, partiell beobachtbaren 2D-Welt löst, gemessen primär an der Erfolgsquote unter Berücksichtigung der resultierenden Pfadeffizienz. Dazu wird eine C++-basierte Simulationsumgebung mit angebundenem Python-Interface für das Training rekurrenter und nicht-rekurrenter Proximal-Policy-Optimization-Modelle (PPO) verwendet. Die Ergebnisse werden gegen eine ungelernete ε -Kompass-Heuristik eingeordnet, um zu prüfen, ob das gelernte Verfahren tatsächlich einen Mehrwert gegenüber einer einfachen Referenz liefert. Wie in Kapitel 5 gezeigt wird, übertrifft die Heuristik bei $\varepsilon = 0,8$ das beste trainierte Modell sowohl in der Erfolgsquote als auch in der Pfadeffizienz. Kapitel 6 ordnet diesen Befund ein und diskutiert mögliche Ursachen für das Verhalten der rekurrenten Policy.

Schlagwörter: Reinforcement Learning, Proximal Policy Optimization, POMDP, Curriculum Learning, prozedurale Weltgenerierung

Abstract

This work investigates which combination of policy architecture and training procedure best solves a navigation task in a procedurally generated, partially observable 2D world, measured primarily by success rate while accounting for the resulting path efficiency. A custom simulation environment implemented in C++ with a Python interface is used to train recurrent and non-recurrent Proximal Policy Optimization (PPO) agents. The trained models are compared against an untrained ε -compass heuristic baseline to assess whether learning provides a genuine advantage over a simple reference policy. As shown in Chapter 5, the heuristic at $\varepsilon = 0.8$ outperforms the best trained model in both success rate and path efficiency. Chapter 6 discusses this finding and possible explanations for the recurrent policy's behaviour.

Keywords: Reinforcement Learning, Proximal Policy Optimization, POMDP, Curriculum Learning, Procedural World Generation

Inhaltsverzeichnis

Eidesstattliche Erklärung	III
Kurzfassung	IV
Abstract	V
Abbildungsverzeichnis	IX
Tabellenverzeichnis	XI
Quelltextverzeichnis	XII
Abkürzungsverzeichnis	XIII
1 Einleitung	1
1.1 Motivation	1
1.2 Problemstellung	1
1.3 Zielsetzung und Forschungsfrage	1
1.4 Vorgehen und Methodik	2
1.5 Abgrenzung	2
1.6 Verwandte Arbeiten	3
2 Grundlagen	4
2.1 Grundlagen der prozeduralen Weltgenerierung	4
2.1.1 Deterministischer Zufall und Pseudozufallszahlen	4
2.1.2 Gradienten- und Value-Noise-Verfahren	4
2.1.3 Zelluläre Automaten zur Gefügeglättung	6
2.1.4 Graphentheoretische Erreichbarkeitsanalyse	7
2.2 Grundlagen des Reinforcement Learning	8
2.2.1 Die Gymnasium-Schnittstelle (ehemals OpenAI Gym)	13
3 Methodik	15
3.1 Methodik der Weltgenerierung	15
3.1.1 Methodische Grundlagen	15
3.1.2 Heuristiken und Regeln	16
3.1.3 Systemkonfiguration	16
3.2 Methodik des RL-Trainings	16
3.2.1 Auswahl der zu vergleichenden Trainingsverfahren	17
3.2.2 Problemformulierung: Zustand, Aktionsraum und Reward	17

3.2.3	Curriculum Konzept und Schnittstelle zur Weltgenerierung	18
3.2.4	Definition der Baselines	18
3.2.5	Trainings- und Evaluationsprotokoll	19
3.2.6	Validierungsstrategie und Umgang mit Trainingsinstabilitäten	20
4	Umsetzung und technische Realisierung	21
4.1	Implementierung der Weltgenerierung	21
4.1.1	Implementierung der Rauscherzeugung und des Koordinaten-Hashings	21
4.1.2	Biomeinteilung und Objekt-Placement	23
4.1.3	Topologische Sicherheits-Fallbacks und Spawn-Freiräume	25
4.1.4	Generierung von Gewässern und dynamische Strand-Derivation	27
4.1.5	Chunk-Speicherarchitektur und prozedurales Biome-Blending	30
4.1.6	Prozedurale Struktur-Platzierung und dynamisches Gegner-Spawning	33
4.1.7	Spieler-Bewegung, Kollisionsabfrage und Kampfdynamik	35
4.2	Implementierung des RL-Trainings	37
4.2.1	Technologie-Stack	37
4.2.2	Anbindung der Spielumgebung und Aufbau der Beobachtung	38
4.2.3	Netzwerkarchitekturen: Asymmetrie zwischen LSTM- und MLP-Policy	39
4.2.4	Trainingspipeline und duales Curriculum-Gating	40
4.2.5	Reward-Shaping und Distanzfeld	42
4.2.6	Umsetzung der Baselines: Die wandblinde Kompass-Heuristik	43
4.2.7	Vollständige Hyperparameter-Konfiguration	44
5	Evaluation	46
5.1	Evaluation der Weltgenerierung	46
5.1.1	Visuelle Ergebnisse und Biome-Gradients	46
5.1.2	Synthese der Spielelemente	46
5.1.3	Artefaktanalyse: Das Phänomen des Biome-Swallowing	47
5.1.4	Lösungsansätze für kleine Biome	47
5.1.5	Performance-Evaluation und Benchmarking	48
5.1.6	Kritische Diskussion und Limitationen	49
5.2	Evaluation des RL-Trainings	50
5.2.1	Erfolgsquote der trainierten Verfahren	50
5.2.2	Pfadeffizienz derselben Verfahren	51
5.2.3	Die Determinismus-Lücke	53
5.2.4	Einordnung gegenüber den Referenzpunkten	54
5.2.5	Wirksamkeit der Hyperparameter	55
6	Fazit und Ausblick	57
6.1	Diskussion der Ergebnisse	57
6.2	Fazit	58
6.2.1	Beitrag der Weltgenerierung (Laurin)	58
6.2.2	Gesamtbild (gemeinsam)	59

6.3	Ausblick	59
6.3.1	Weltgenerierung (Laurin)	60
6.3.2	Reinforcement Learning (Florian)	60
	Literaturverzeichnis	62
	A Anhang	64

Abbildungsverzeichnis

2.1	Value Noise – lineare gegen Smoothstep-Interpolation zwischen denselben Knotenwerten, eigene Darstellung.	5
2.2	Domain Warping – derselbe Rauschausschnitt unverzerrt und verzerrt, eigene Darstellung.	6
2.3	Zellulärer Automat – Rauschgitter vor und nach vier Glättungsiterationen, eigene Darstellung.	7
2.4	Breitensuche – Distanzfeld vom Startpunkt und daraus resultierender kürzester Pfad, eigene Darstellung.	8
2.5	Interaktionsschleife zwischen Agent und Umgebung.	9
2.6	Vergleich zwischen vollständiger und partieller Beobachtbarkeit.	9
2.7	Veranschaulichung des Potentialfelds $\Phi(s)$ und der daraus resultierenden formenden Belohnung entlang eines Beispielpfads.	10
2.8	Darstellung der Clipped Surrogate Objective von PPO.	11
2.9	Funktionsweise einer rekurrenten Policy im Vergleich zu einer zustandslosen Policy.	12
2.10	Zusammenspiel von Replay Buffer und Target Network in der DQN Architektur.	13
3.1	Struktur des vierphasigen Curriculum-Trainings mit Steigerung der Exit-Distanz	18
3.2	Entscheidungsregel und ausgewertete Parameter der ε -Kompass-Heuristik .	19
4.1	Nahaufnahme einer prozedural generierten Seenstruktur mit organischer, rauschbasierter Uferlinie und dynamisch abgeleitetem Strand-Overlay an den Begehrkeitsgrenzen. Quelle: Eigene Darstellung (Screenshot des Prototyps).	29
4.2	Vergleichende Darstellung von vier der sieben implementierten Biome anhand von Screenshots des Prototyps. Die unterschiedlichen Untergrund-Farbpaletten sowie Objekt- und Hindernisdichten ergeben sich aus den biomspezifischen Schwellenwerten (<code>wallThreshold</code> , <code>oreThreshold</code> , <code>treeThreshold</code>) aus Abschnitt 4.1.2. Quelle: Eigene Darstellung (Screenshots des Prototyps). 31	
5.1	Visualisierung der generierten Spielwelt mit dynamischer Beleuchtung/Sichtfeld, organischen Seenstrukturen, Strandbereichen und biomabhängigen Kacheltypen. Quelle: Eigene Darstellung (Screenshot des Prototyps, Debug-Overlay entfernt).	46

5.2	Evaluation – Erfolgsquote der trainierten Verfahren und der Referenzpunkte auf Testset A, eigene Darstellung.	51
5.3	Evaluation – Pfadeffizienz der Verfahren in derselben Achsenordnung, eigene Darstellung.	52
5.4	Evaluation – Erfolgsquote desselben Modellstands in deterministischer und stochastischer Auswertung, eigene Darstellung.	53
5.5	Evaluation – Erfolgsquote gegen Pfadeffizienz für alle Verfahren, eigene Darstellung.	54

Tabellenverzeichnis

3.1	Konfigurationsstatus der Weltengenerierung	16
3.2	Übersicht der POMDP-Komponenten für das RL-Training	18
3.3	Diagnose- und Interventionsmatrix der aufgetretenen Trainingsinstabilitäten	20
4.1	Eingesetzte Werkzeuge des RL-Trainings	38
4.2	Allgemeine Hyperparameter des PPO-Optimierers	45
5.1	Übersicht der Performancemessungen und Benchmarks des Generierungs- und Simulationssystems.	48
5.2	Belegte Wirkung einzelner Parameterentscheidungen	55

Quelltextverzeichnis

4.1	Verwendung des 64-Bit SplitMix-Mixers und Value-Noise-Interpolation in <code>world.cpp</code>	21
4.2	Biom-Tagging und Schwellenwertauswertung in <code>world.cpp</code>	23
4.3	Freiradien-Erzeugung und Manhattan-Path-Carver in <code>world.cpp</code>	25
4.4	Low-Frequency-Value-Noise für zusammenhängende Gewässermasken in <code>world.cpp</code>	27
4.5	Nachbarschaftsbasierte Strandardarstellung im Render-Pfad in <code>render_engine.cpp</code>	28
4.6	Chunk-Speicherstruktur und Lazy-Loading-Mechanismus in <code>world.hpp</code> und <code>world.cpp</code>	30
4.7	Gewichtete Biom-Mischung und Tinting im Render-Pfad (<code>render_engine.cpp</code>)	31
4.8	Biomabhängige Struktur-Platzierung bei der Chunk-Generierung in <code>world.cpp</code>	33
4.9	Lazy Mob-Spawning und lokale Bewegungslogik in <code>simulation.cpp</code>	34
4.10	Diskrete Bewegungs- und Kollisionsprüfung in <code>simulation.cpp</code> und <code>object.cpp</code>	35
4.11	Flächenangriff des Spielers und Schadensabwicklung in <code>simulation.cpp</code> . .	36
4.12	Aufbau des Beobachtungsvektors, <code>python/stoneforge_env.py</code>	38
4.13	Instanziierung der Modelle mit asymmetrischen Parametern, <code>scripts/train_curriculum.py</code>	39
4.14	Curriculum-Phasen mit dualen Abbruchbedingungen, <code>scripts/train_curriculum.py</code>	41
4.15	Berechnung des Rewards und Shaping-Terms, <code>src/core/simulation.cpp</code> .	42
4.16	Aktionswahl der Referenzverfahren, <code>scripts/eval_baselines.py</code>	43

Abkürzungsverzeichnis

Abkürzung	Bedeutung
A2C	Advantage Actor-Critic
API	Application Programming Interface
ASCII	American Standard Code for Information Interchange
BFS	Breitensuche (Breadth-First Search)
CA	Zelluläre Automaten (Cellular Automaton)
CLI	Command Line Interface
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture
DQN	Deep Q-Network
FPS	Frames per Second
KI	Künstliche Intelligenz
LRU	Least Recently Used
LSTM	Long Short-Term Memory
MDP	Markov Decision Process
MLP	Multi-Layer Perceptron
MPS	Metal Performance Shaders
PBRs	Potential-Based Reward Shaping
PCGRL	Procedural Content Generation via Reinforcement Learning
POMDP	Partially Observable Markov Decision Process
PPO	Proximal Policy Optimization
PRNG	Pseudo-Random Number Generator
RAM	Random Access Memory
RL	Reinforcement Learning
RNN	Recurrent Neural Network
SR	Success Rate
TRPO	Trust Region Policy Optimization

1 Einleitung

1.1 Motivation

Die Idee zu dieser Projektarbeit entstand aus den unterschiedlichen Interessen im Team: Laurin hat sich schon länger für Spieleentwicklung begeistert, während Florians Schwerpunkt eher im Bereich Machine Learning und Reinforcement Learning lag. Da lag es nahe, beide Themen miteinander zu verbinden.

Als Vorbild dienten Spiele wie *Minecraft*, deren Reiz zu einem großen Teil darin liegt, dass Welten bei jedem Start prozedural neu erzeugt werden. Das Ziel war es, ein ähnliches Prinzip im Kleinen umzusetzen: Eine einfache, prozedural generierte Spielwelt zu bauen und darauf einen RL-Agenten zu trainieren, der lernen muss, sich in dieser unbekannten Umgebung zurechtzufinden und den Ausgang zu erreichen.

1.2 Problemstellung

Das zentrale Problem in der Spielwelt ist, dass der Agent zu keinem Zeitpunkt die gesamte Karte sieht. Er bekommt immer nur einen kleinen Ausschnitt direkt um seine eigene Position herum angezeigt und muss trotzdem versuchen, den Ausgang zu finden, der irgendwo im unentdeckten Bereich liegt.

Diese Einschränkung wurde bewusst eingebaut: Wenn der Agent immer die komplette Karte auf einmal sehen könnte, wäre die Aufgabe im Grunde viel zu einfach. Das ließe sich dann schon mit ein paar festen Regeln oder einfachen Algorithmen lösen, ohne dass ein RL-Agent wirklich etwas lernen müsste.

Durch die eingeschränkte Sicht muss der Agent jedoch mit unvollständigem Wissen arbeiten, etwa so wie ein Mensch, der sich ohne Karte durch ein völlig unbekanntes Gebäude tastet. Der Agent muss also durch eine ihm unbekannte Welt navigieren, die er erst beim Durchlaufen Schritt für Schritt überhaupt erst entdeckt.

1.3 Zielsetzung und Forschungsfrage

Das Ziel der Arbeit ist es, eine eigene prozedurale 2D-Spielwelt zu bauen und darauf verschiedene RL-Agenten für eine Navigationsaufgabe mit eingeschränkter Sicht zu trainieren.

Daraus ergibt sich die zentrale Hauptfrage: Wie muss eine prozedural generierte 2D-Spielwelt aufgebaut sein und welche RL-Algorithmen erzielen darin die besten Ergebnisse bei der Navigation?

Konkret unterteilen wir das in zwei Teilfragen:

- **Weltdesign:** Welche Parameter (wie Wanddichte oder Kartengröße) machen die Welt fair und für den Agenten lösbar?
- **Algorithmen & Parameter:** Welches RL-Verfahren erreicht die höchste Erfolgsquote und die besten Wege zum Ausgang?

1.4 Vorgehen und Methodik

Unser Vorgehen ist in vier Schritte aufgeteilt:

1. **Recherche zu Weltgenerierung und RL-Algorithmen:** Zuerst recherchieren wir, wie eine prozedurale Generierung sinnvoll aufgebaut sein sollte und welche Algorithmen sich eignen, um zufällige, aber immer lösbare 2D-Welten zu erzeugen. Parallel dazu schauen wir uns an, welche RL-Algorithmen für Navigationsaufgaben mit eingeschränkter Sicht überhaupt infrage kommen.
2. **Prototyping der Spielumgebung:** Auf dieser Basis bauen wir einen ersten Prototyp der Spielwelt. Diesen verbessern wir iterativ, bis Parameter wie Kartengröße, Wanddichte und die Sichtweite des Agenten gut zusammenpassen und die Level fair bleiben.
3. **Training der RL-Agenten:** Sobald die Spielumgebung stabil steht, setzen wir die ausgewählten RL-Algorithmen darauf an, trainieren sie und passen die Hyperparameter Schritt für Schritt an.
4. **Evaluation und Auswertung:** Zum Schluss testen wir die fertig trainierten Agenten auf vielen neu generierten Leveln. Wir erfassen die Ergebnisse – wie die Erfolgsquote und die Pfadeffizienz – systematisch und vergleichen die Verfahren miteinander.

1.5 Abgrenzung

Damit der Rahmen der Projektarbeit überschaubar bleibt, grenzen wir folgende Punkte bewusst ab:

- **Fokus auf die eigene Spielwelt:** Wir untersuchen ausschließlich unsere selbst entwickelte 2D-Gitterwelt. Eine Übertragbarkeit auf 3D-Umgebungen oder andere externe Benchmarks testen wir nicht.
- **Kein RL für die Weltgenerierung:** Reinforcement Learning wird bei uns nur für das Navigationsverhalten des Agenten eingesetzt. Die Spielwelt selbst wird über feste, klassische Algorithmen erzeugt (kein gelerntes Weltdesign).
- **Begrenzte Parameterstudie:** Wir führen keine unendliche Testreihe über alle denkbaren Kombinationen aus Wanddichten oder Kartengrößen durch, sondern konzentrieren uns auf die ausgewählten RL-Verfahren und praxisnahe Welteinstellungen.

1.6 Verwandte Arbeiten

Unser Ansatz lässt sich in bestehende Arbeiten einordnen, die sich mit gitterbasierten Spielwelten und prozeduraler Generierung für Reinforcement Learning beschäftigen:

- **Gitterbasierte Testwelten (MiniGrid & Crafter):** Umgebungen wie *MiniGrid* [3] oder *Crafter* [5] verfolgen ein ganz ähnliches Prinzip wie unser Projekt. Sie bieten leichtgewichtige 2D-Gitterwelten, in denen der Agent oft nur einen begrenzten Ausschnitt der Karte sieht und einfache Navigations- oder Sammelaufgaben lösen muss.
- **Generalisierung durch prozedurale Level (Procgen):** Der *Procgen Benchmark* [4] untersucht gezielt, wie gut RL-Agenten auf immer wieder neu generierten Levels verallgemeinern. Das bestätigt unsere Grundidee, dass prozedurale Welten verhindern, dass ein Agent feste Pfade einfach nur auswendig lernt.
- **Partielle Beobachtbarkeit und Gedächtnis (POPGym):** Besonders nah an unserem RL-Teil ist *POPGym* [11]. Die Autoren untersuchen dort ganz gezielt Umgebungen mit eingeschränkter Sicht (partielle Beobachtbarkeit) und vergleichen, wie sich Modelle mit Gedächtnis (rekurrente Policies) gegen Modelle ohne Gedächtnis schlagen. Das ist exakt der Ansatz, den wir auch für den Vergleich unserer Trainingsverfahren übernehmen.
- **Abgrenzung zu gelerntem Weltdesign (PCGRL):** In Arbeiten wie *PCGRL* [9] wird Reinforcement Learning eingesetzt, um das Generieren von Spielwelten selbst zu erlernen. Davon grenzen wir uns klar ab: Bei uns bleibt die Erzeugung der Spielwelt fest über eigene Algorithmen programmiert, und das RL-Verfahren dient ausschließlich dazu, das Navigationsverhalten des Agenten zu trainieren.

2 Grundlagen

Dieses Kapitel legt die theoretischen und mathematischen Grundlagen für beide Teile der Arbeit. Zunächst behandelt Abschnitt 2.1 die Grundlagen der prozeduralen Weltgenerierung, von deterministischem Zufall über Rauschverfahren bis zur graphentheoretischen Erreichbarkeitsanalyse. Im Anschluss folgen in Abschnitt 2.2 die Grundlagen des Reinforcement Learning.

2.1 Grundlagen der prozeduralen Weltgenerierung

Unter prozeduraler Generierung versteht man Verfahren, bei denen digitale Inhalte nicht manuell durch Designer erstellt, sondern algorithmisch zur Laufzeit berechnet werden. Dies ermöglicht die Erzeugung sehr großer oder theoretisch unendlicher Spielwelten bei geringem Speicherbedarf. Die folgenden Abschnitte bauen aufeinander auf: Zunächst wird die deterministische Zufallsgrundlage behandelt (Abschnitt 2.1.1), darauf aufbauend die daraus abgeleiteten Rauschverfahren (Abschnitt 2.1.2) und deren nachträgliche Glättung (Abschnitt 2.1.3). Den Abschluss bildet die graphentheoretische Absicherung der Spielbarkeit (Abschnitt 2.1.4).

2.1.1 Deterministischer Zufall und Pseudozufallszahlen

In der Informatik ist echter Zufall (*True Randomness*) ohne spezielle Hardware-Schnittstellen schwer zu realisieren. Stattdessen kommen Pseudozufallszahlengeneratoren (*Pseudo-Random Number Generators*, PRNG) zum Einsatz. Ein wesentliches Merkmal für die Weltgenerierung ist der Determinismus:

- **Seed (Startwert):** Ein PRNG erzeugt eine deterministische Zahlenfolge basierend auf einem initialen Eingabewert (Seed). Bei identischem Seed erzeugt der Algorithmus jedes Mal exakt dieselbe Zahlenfolge.
- **Hash- und Mix-Funktionen:** Um räumliche Daten (wie zweidimensionale Koordinaten (x, y)) unabhängig von einem fortlaufenden Zustand abzufragen, werden häufig zustandslose Hash-Funktionen genutzt. Diese bilden Raumkoordinaten, den Seed und ein optionales *Salt* (ein zusätzlicher Offset zur Unterscheidung verschiedener Abfragen) auf einen pseudozufälligen Wert im Intervall $[0, 1]$ ab.

2.1.2 Gradienten- und Value-Noise-Verfahren

Die direkte Verwendung unkorrelierter Pseudozufallswerte für benachbarte Raumpunkte führt zu einer ungeordneten Struktur (weißes Rauschen). Für die Simulation natürlicher

Landschaften werden daher *Coherent-Noise*-Verfahren genutzt, bei denen nahe beieinander liegende Punkte ähnliche Werte aufweisen.

Value Noise und Interpolation

Beim *Value Noise* werden an den Knotenpunkten eines regulären Gitters pseudozufällige Skalarwerte bestimmt. Die Werte für Punkte innerhalb einer Gitterzelle berechnen sich anschließend durch Interpolation der umliegenden Knotenwerte.

Um stetige Übergänge ohne harte Kanten an den Zellgrenzen zu gewährleisten, wird der Interpolationsparameter $t \in [0, 1]$ vor der linearen Interpolation (*Lerp*) durch eine S-förmige Glättungsfunktion (*Smoothstep*) transformiert:

$$S(t) = 3t^2 - 2t^3 \quad (2.1)$$

Abbildung 2.1 stellt beide Interpolationsvarianten an denselben pseudozufälligen Knotenwerten gegenüber. Der Unterschied zeigt sich am deutlichsten an den Knoten selbst: Die lineare Variante bildet dort sichtbare Knicke, während die Smoothstep-Kurve glatt hindurchläuft.

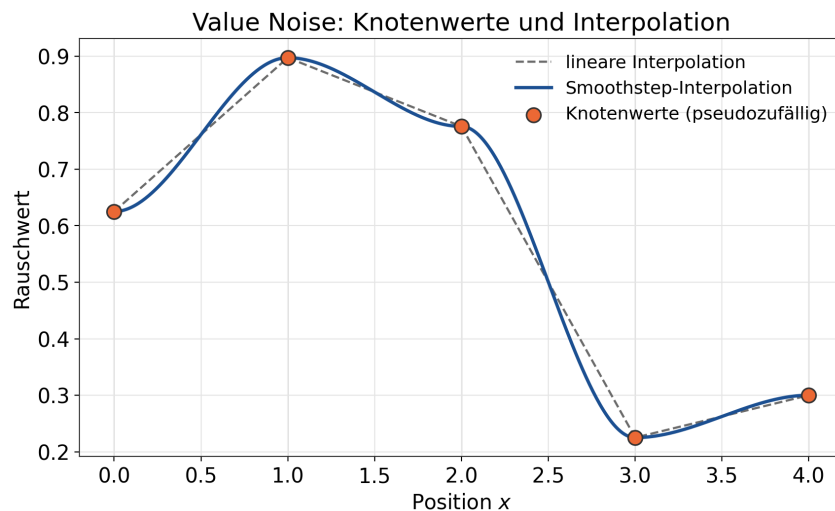


Abbildung 2.1: Value Noise – lineare gegen Smoothstep-Interpolation zwischen denselben Knotenwerten, eigene Darstellung.

Im zweidimensionalen Raum erfolgt die Überlagerung der Interpolationen entlang beider Achsen mittels bilinearer Interpolation.

Domain Warping und fraktale Akkumulation

Um die typischen Raster-Artefakte einfacher Gitterstrukturen zu vermeiden, werden fortgeschrittene Techniken eingesetzt:

- **Multi-Frequency / Fraktaler Noise:** Durch die Addition mehrerer Rauschfunktionen unterschiedlicher Frequenzen (Oktaven) und Amplituden lassen sich fraktal-ähnliche, hochdetaillierte Strukturen erzeugen. Niedrige Frequenzen steuern groß-

flächige Formen (z. B. Kontinente), während hohe Frequenzen feine Details (z. B. Felsstrukturen) ergänzen.

- **Domain Warping:** Hierbei werden die Eingangskordinaten (x, y) einer Rauschfunktion selbst durch den Ausgangswert einer anderen Rauschfunktion verzerrt:

$$f_{\text{warped}}(x, y) = f(x + g(x, y), y + h(x, y)) \quad (2.2)$$

Dies führt zu organischen Verformungen und eliminiert visuelle Gitterlinien.

Abbildung 2.2 zeigt denselben Rauschausschnitt mit und ohne Domain Warping. Die verzerrte Variante rechts löst die diagonalen Vorzugsrichtungen des unverzerrten Rauschens auf und wirkt dadurch weniger regelmäßig.

Domain Warping einer Rauschfunktion

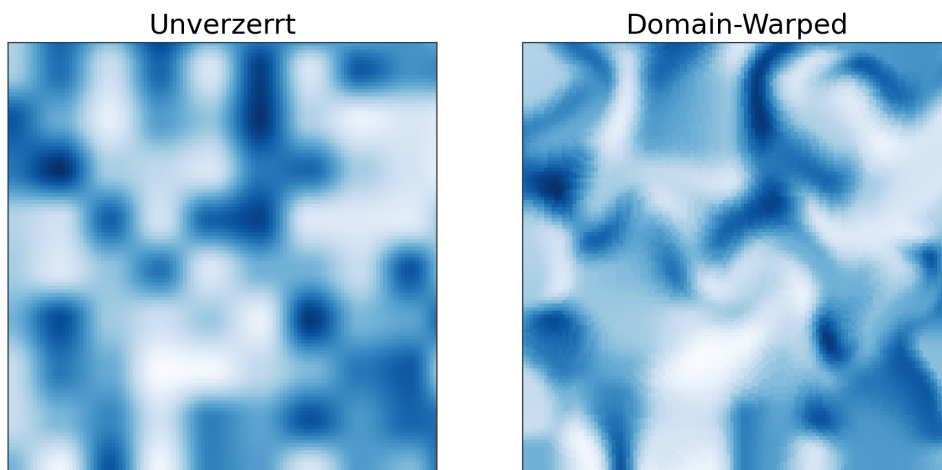


Abbildung 2.2: Domain Warping – derselbe Rauschausschnitt unverzerrt und verzerrt, eigene Darstellung.

2.1.3 Zelluläre Automaten zur Gefügeglättung

Zelluläre Automaten (*Cellular Automata*) bestehen aus Zellen, die diskrete Zustände (z. B. *Wand* oder *Freiraum*) annehmen können. In diskreten Zeitschritten aktualisiert jede Zelle ihren Zustand basierend auf einer lokalen Übergangsfunktion, welche die Zustände ihrer Nachbarschaft berücksichtigt.

Häufig wird hierbei die Moore-Nachbarschaft verwendet, welche die 8 direkt angrenzenden Zellen (horizontal, vertikal und diagonal) umfasst. Durch gezielte Geburts- und Überlebensregeln (*Birth/Survival Rules*) lassen sich aus verrauschten Initialzuständen zusammenhängende, natürliche Höhlen- und Klippenstrukturen erzeugen. Abbildung 2.3 zeigt diesen Effekt an einem unabhängigen Beispielgitter mit der auch in der Projektkonfiguration hinterlegten Regel B5/S4 (Geburt ab 5, Überleben ab 4 Nachbarwänden): Aus unstrukturiertem Rauschen entstehen nach wenigen Iterationen zusammenhängende Flächen.

Im ausgelieferten Konfigurationsstand ist diese Nachbearbeitung deaktiviert (siehe Tabelle 3.1). Sie wird hier dennoch als Grundlage behandelt, weil sie die naheliegende algo-

rhythmische Alternative zum tatsächlich verwendeten Manhattan-Fallback markiert (siehe Abschnitt 2.1.4) und dessen Auswahl erst vor diesem Hintergrund einordbar wird.

Zelluläre Automaten zur Gefügeglättung



Abbildung 2.3: Zellulärer Automat – Rauschgitter vor und nach vier Glättungsiterationen, eigene Darstellung.

2.1.4 Graphentheoretische Erreichbarkeitsanalyse

Ein zentrales Kriterium prozeduraler Welten ist die Sicherstellung der Spielbarkeit, insbesondere die Vermeidung unlösbarer Level-Zustände (*Deadlocks*).

Breitensuche (Breadth-First Search, BFS)

Zur Analyse der Konnektivität wird das Zellgitter als ungerichteter Graph interpretiert, in dem begehbare Zellen als Knoten und Nachbarschaftsbeziehungen als Kanten dargestellt werden. Die Breitensuche (BFS) traversiert den Graphen ebenenweise vom Startknoten aus. Sie eignet sich zur Prüfung der Erreichbarkeit eines Zielknotens sowie zur Bestimmung des kürzesten Pfades auf ungewichteten Gittern. Abbildung 2.4 veranschaulicht das resultierende Distanzfeld an einem Beispielgitter: Jede Ebene entspricht einem BFS-Schritt vom Start aus, der kürzeste Pfad zum Ziel ergibt sich durch Rückverfolgung der jeweils näher liegenden Nachbarzelle.

Im ausgelieferten Konfigurationsstand dient die Breitensuche nicht mehr der Lösbarkeitsprüfung bei der Weltgenerierung (siehe Tabelle 3.1), da diese Rolle vollständig vom Manhattan-Fallback übernommen wird. Sie bleibt jedoch an anderer Stelle aktiv: als Distanzfeld für das Reward Shaping im RL-Training (Abschnitt 4.2.5) und als Bezugsgröße der Pfadeffizienz-Metrik in der Evaluation (Abschnitt 5.2.2).

Heuristische Validierung und Korrekturverfahren

Da die exakte graphenbasierte Validierung bei großen Chunks rechenintensiv sein kann, werden in der Praxis häufig kombinierte Ansätze gewählt:

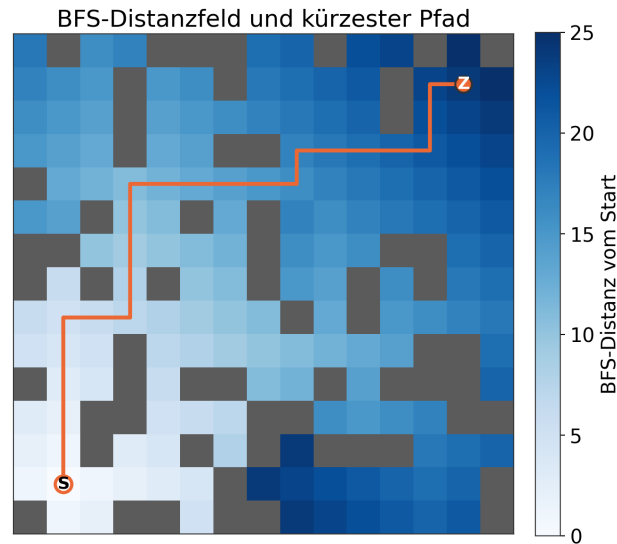


Abbildung 2.4: Breitensuche – Distanzfeld vom Startpunkt und daraus resultierender kürzester Pfad, eigene Darstellung.

- **Manhattan-Distanz:** Die Distanz zwischen zwei Punkten (x_1, y_1) und (x_2, y_2) auf einem orthogonalen Gitter berechnet sich nach:

$$D_{\text{Manhattan}} = |x_1 - x_2| + |y_1 - y_2| \quad (2.3)$$

- **Fallback-Generierung (Carving):** Sollte keine valide Verbindung existieren, stellen geometrische Ausgrabungsalgorithmen (*Carving*) entlang direkter Achsen eine funktionale Wegeverbindung sicher.

2.2 Grundlagen des Reinforcement Learning

Das Reinforcement Learning (RL) ist ein Teilbereich des maschinellen Lernens, bei dem ein Agent lernt, durch Interaktion mit einer Umgebung optimale Entscheidungen zu treffen (siehe Abbildung 2.5). Die formale mathematische Grundlage hierfür bildet der **Markov Decision Process (MDP)**. Ein MDP wird typischerweise als Tupel (S, A, P, R, γ) definiert. Dabei steht S für die Menge aller möglichen Zustände der Umgebung und A für die Menge der möglichen Aktionen des Agenten. Die Übergangswahrscheinlichkeit $P(s' | s, a)$ beschreibt die Dynamik der Umgebung, also mit welcher Wahrscheinlichkeit der Agent durch die Aktion a vom Zustand s in den Folgezustand s' gelangt. Für jeden dieser Übergänge erhält der Agent ein Feedback in Form eines numerischen Belohnungssignals, dem **Reward** $R(s, a)$. Das übergeordnete Ziel des Agenten ist es, eine Verhaltensregel, die sogenannte **Policy** $\pi(a | s)$, zu erlernen. Diese Policy ordnet jedem Zustand eine Aktion zu und maximiert dabei die erwartete, kumulierte Belohnung über die Zeit. Der Diskontierungsfaktor $\gamma \in [0, 1]$ bestimmt dabei, wie stark zukünftige Belohnungen im Vergleich zu unmittelbaren gewichtet werden [16].

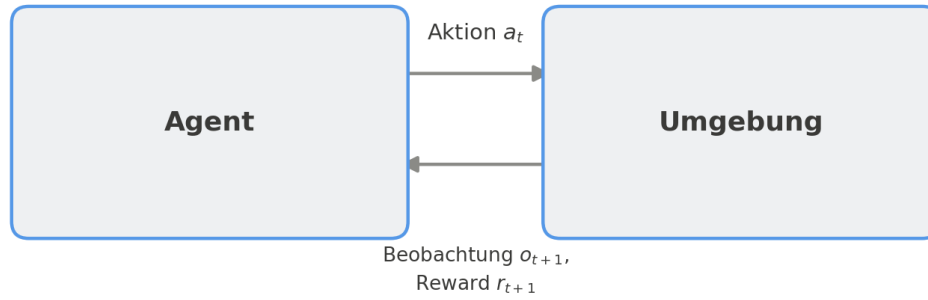


Abbildung 2.5: Interaktionsschleife zwischen Agent und Umgebung.

Partially Observable Markov Decision Processes (POMDP) Die Standarddefinition des MDP geht von vollständiger Beobachtbarkeit aus. Der Agent kennt somit stets den exakten aktuellen Zustand s der Umgebung. In vielen realen Szenarien ist diese Annahme jedoch nicht erfüllt, da Sensordaten verrauscht oder Informationen verborgen sein können. Für solche Problemstellungen wird das Konzept zum *Partially Observable Markov Decision Process* (POMDP) erweitert. In einem POMDP hat der Agent keinen direkten Zugriff auf den wahren Zustand $s \in S$, sondern erhält lediglich eine Beobachtung $o \in \Omega$, die gemäß einer Beobachtungswahrscheinlichkeit $O(o | s, a)$ erzeugt wird (siehe Abbildung 2.6). Um dennoch optimale Entscheidungen treffen zu können, muss der Agent einen sogenannten „Belief State“ (Überzeugungszustand) pflegen. Dieser stellt eine Wahrscheinlichkeitsverteilung über alle möglichen wahren Zustände der Umgebung dar und wird nach jeder neuen Aktion und Beobachtung kontinuierlich aktualisiert [8].

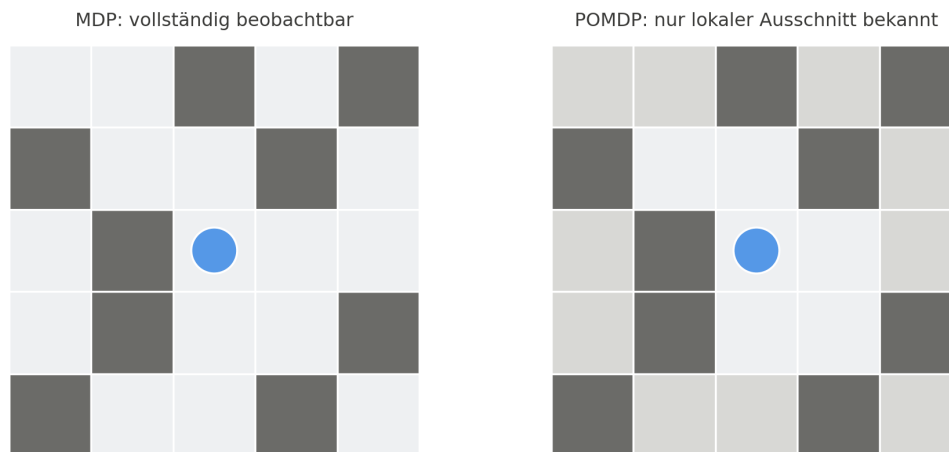


Abbildung 2.6: Vergleich zwischen vollständiger und partieller Beobachtbarkeit.

Potential Based Reward Shaping (PBRS) Eine zentrale Herausforderung im Reinforcement Learning sind verzögerte oder spärliche Belohnungen (Sparse Rewards), bei denen der Agent nur sehr selten ein positives Feedback erhält. Um den Lernprozess zu beschleunigen, wird häufig *Reward Shaping* angewendet, bei dem das ursprüngliche Belohnungssignal durch zusätzliche, dichtere Zwischenbelohnungen angereichert wird. Die allgemeine Theorie des *Potential Based Reward Shaping* (PBRS) löst dabei das Problem, dass naive Anpassungen der Belohnungsfunktion das eigentliche Ziel des Agenten verfälschen könnten.

(sogenanntes Reward Hacking). PBRs stellt mathematisch sicher, dass die optimale Policy des ursprünglichen MDPs trotz der zusätzlichen Belohnungen strikt erhalten bleibt. Dies wird erreicht, indem die formende Belohnung $F(s, a, s')$ als Differenz einer Potenzialfunktion Φ zwischen dem Folgezustand und dem aktuellen Zustand definiert wird (vergleiche Abbildung 2.7): $F(s, a, s') = \gamma\Phi(s') - \Phi(s)$. Diese Funktion gibt dem Agenten kontinuierliches Feedback basierend auf seinem „Abstand“ zum Ziel [13].

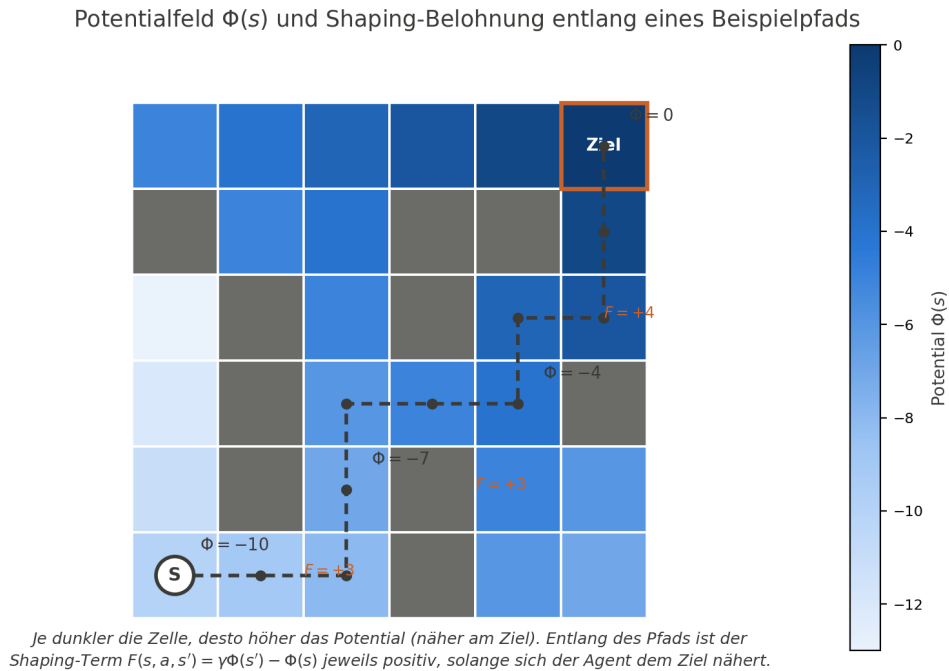


Abbildung 2.7: Veranschaulichung des Potentialfelds $\Phi(s)$ und der daraus resultierenden formenden Belohnung entlang eines Beispielpfads.

Curriculum Learning Wenn Aufgaben für einen untrainierten RL Agenten zu komplex sind, schlägt das Lernen oft vollständig fehl, da der Agent rein durch zufällige Exploration keine Lösung findet. Das allgemeine Prinzip des *Curriculum Learning* greift dieses Problem auf, indem es sich an der menschlichen Pädagogik orientiert: Anstatt dem Agenten sofort die finale, hochkomplexe Aufgabe zu stellen, wird der Lernprozess strukturiert. Der Agent trainiert zunächst auf stark vereinfachten Versionen der Aufgabe. Sobald er diese beherrscht, wird der Schwierigkeitsgrad der Umgebung oder der Aufgabenstellung schrittweise erhöht, bis die Zieldomäne erreicht ist. Diese methodische Abfolge (das Curriculum) beschleunigt nicht nur die Konvergenz des Lernalgorithmus erheblich, sondern ermöglicht es oft erst, dass der Agent für sehr komplexe Problemstellungen überhaupt eine optimale Policy findet [1, 12].

Proximal Policy Optimization (PPO) Proximal Policy Optimization (PPO) gehört zur Familie der Policy Gradient Methoden und wurde von Schulman et al. [15] als Weiterentwicklung etablierter Verfahren wie der Trust Region Policy Optimization (TRPO) eingeführt. Der zentrale Vorteil von PPO liegt in der Balance zwischen einfacher Implementierung, hoher Stichprobeneffizienz (Sample Efficiency) und Trainingsstabilität. Während

herkömmliche Methoden des Policy Gradient bei zu großen Aktualisierungsschritten destabilisieren und die erlernte Verhaltensregel ruinieren können, führt PPO eine sogenannte Clippingfunktion (*Clipped Surrogate Objective*) ein (vergleiche Abbildung 2.8). Diese Zielfunktion beschränkt die Änderungen der Policy zwischen zwei aufeinanderfolgenden Trainingsschritten auf ein definiertes Intervall, meist parametrisiert durch einen Hyperparameter ϵ (zum Beispiel $[1 - \epsilon, 1 + \epsilon]$). Dadurch wird mathematisch verhindert, dass der Agent seine Strategie in einem einzelnen Updateschritt zu drastisch ändert. Dies führt zu einer monotoneren Verbesserung und einem wesentlich robusteren Lernverhalten, ohne den hohen rechnerischen Aufwand, der bei TRPO für die Berechnung der Kullback Leibler Divergenz anfällt [15].

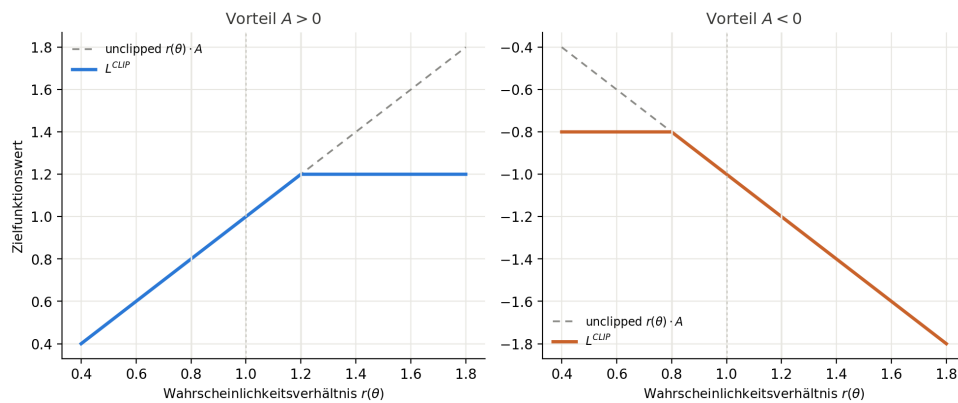


Abbildung 2.8: Darstellung der Clipped Surrogate Objective von PPO.

Rekurrente Neuronale Netze (RNN) Wie in der Definition der POMDPs beschrieben, kann ein Agent in Umgebungen mit partieller Beobachtbarkeit den wahren Zustand nicht aus einer einzelnen, isolierten Beobachtung ableiten. Um dieses Problem zu überwinden, muss der Agent auf vergangene Beobachtungen zurückgreifen können, wofür rekurrente neuronale Netze (RNNs) eingesetzt werden. Im Gegensatz zu zustandslosen Netzwerken (Feedforward Netzen) besitzen RNNs einen internen Speicher, der bei jedem Zeitschritt basierend auf der aktuellen Eingabe und dem vorherigen Zustand aktualisiert wird. Durch diesen impliziten Gedächtnisaufbau kann das Netzwerk die verborgenen Zustandsinformationen eines POMDPs aus der Historie der Beobachtungen näherungsweise rekonstruieren. Eine rekurrente Policy ist somit in der Lage, auch dann optimale Entscheidungen zu treffen, wenn das aktuelle Beobachtungssignal allein keine eindeutige Zustandsschätzung zulässt [6]. Ein grundlegendes Problem einfacher RNNs ist jedoch das Phänomen des verschwindenden Gradienten (*Vanishing Gradient*), wodurch das Netzwerk große Schwierigkeiten hat, Abhängigkeiten über längere Zeiträume stabil zu erlernen.

Long Short Term Memory (LSTM) Um das Problem des verschwindenden Gradienten zu lösen, bildet die *Long Short Term Memory* Architektur die fundamentale Basis für moderne rekurrente Netze. Sie wurde von Hochreiter and Schmidhuber [7] entwickelt und stellt eine spezialisierte Form von RNNs dar. LSTMs nutzen ein System aus dedizierten Speicherzellen und Gatingmechanismen (Input Gates, Output Gates und Forget Gates).

Diese Struktur ermöglicht es dem Netzwerk gezielt zu steuern, welche relevanten Informationen über lange Zeitreihen hinweg gespeichert, aktualisiert oder vergessen werden sollen. Für das Reinforcement Learning ist dies von entscheidender Bedeutung: Der Agent kann dadurch wichtige Ereignisse, die viele Schritte in der Vergangenheit liegen, verlustfrei in seinem Gedächtnis behalten und für aktuelle Entscheidungen heranziehen (siehe Abbildung 2.9). Die Integration von LSTM Zellen ermöglicht es dem Agenten somit, Sequenzen von Beobachtungen über die Zeit effektiv zu aggregieren und komplexe Aufgaben in unvollständig beobachtbaren Umgebungen erfolgreich zu lösen [6, 7].

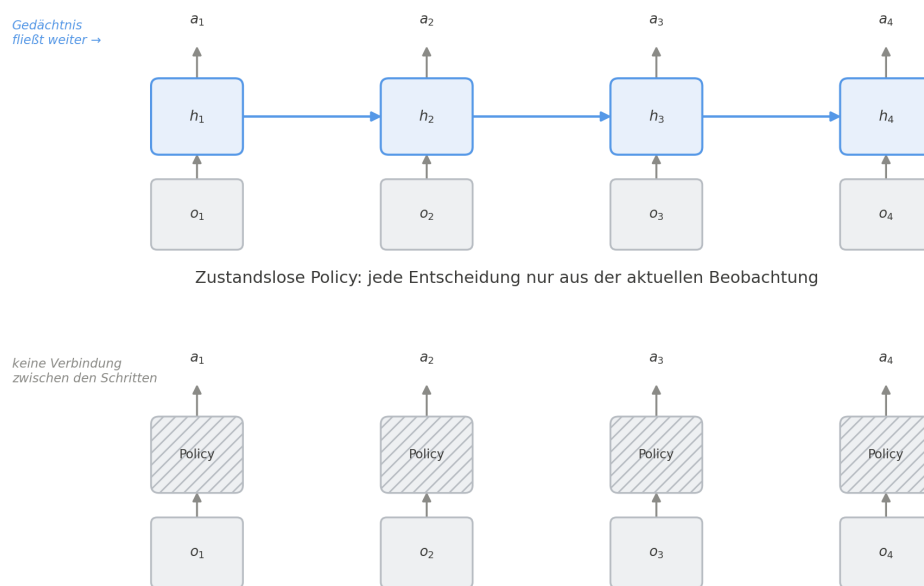


Abbildung 2.9: Funktionsweise einer rekurrenten Policy im Vergleich zu einer zustandslosen Policy.

Deep Q Networks (DQN) Deep Q Networks (DQN) stellen einen entscheidenden Meilenstein in der Entwicklung des modernen Reinforcement Learning dar und wurden maßgeblich durch Mnih et al. [10] geprägt. Im Gegensatz zu den zuvor beschriebenen Methoden des Policy Gradient handelt es sich bei DQN um einen wertebasierten Ansatz. Anstatt die Policy direkt zu parametrisieren, approximiert ein tiefes neuronales Netz die sogenannte Q Funktion (Action Value Funktion). Diese Funktion schätzt die erwartete kumulierte Belohnung für eine spezifische Aktion in einem bestimmten Zustand. Da die Kombination von klassischem Q Learning mit nichtlinearen Funktionsapproximatoren wie neuronalen Netzen historisch oft zu starker Instabilität und Divergenz im Lernverhalten führte, führte DQN zwei zentrale Innovationen ein (siehe Abbildung 2.10). Zum einen das *Experience Replay*, bei dem vergangene Interaktionen des Agenten kontinuierlich in einem Ringpuffer gespeichert und während des Trainings in zufälligen Batches wiederverwendet werden. Dies bricht die zeitliche Korrelation der Trainingsdaten auf und glättet die Datenverteilung. Zum anderen wird ein separates *Target Network* genutzt, dessen Parameter im Gegensatz zum primären Netzwerk nur periodisch aktualisiert werden. Dies hält die Zielwerte für die Fehlerberechnung über mehrere Trainingsschritte hinweg konstant und

verhindert Aufschaukelungseffekte. Durch diese beiden Mechanismen ermöglichte DQN erstmals das stabile Erlernen komplexer Verhaltensstrategien direkt aus hochdimensionalen Beobachtungen [10].

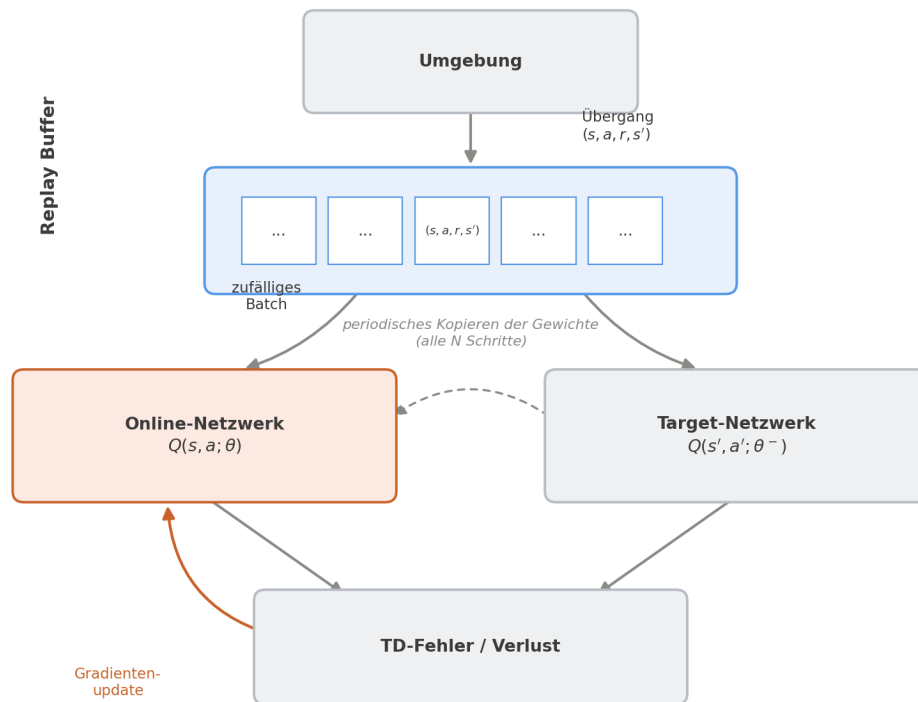


Abbildung 2.10: Zusammenspiel von Replay Buffer und Target Network in der DQN Architektur.

2.2.1 Die Gymnasium-Schnittstelle (ehemals OpenAI Gym)

Um Reinforcement-Learning-Algorithmen unabhängig von der konkreten Problemstellung entwickeln und testen zu können, bedarf es einer standardisierten Kommunikationsschicht zwischen dem lernenden Agenten und der simulierten Welt. In der Forschung und Industrie hat sich hierfür ein dominanter Standard etabliert, der ursprünglich 2016 von OpenAI unter dem Namen „Gym“ veröffentlicht wurde [2]. Um die langfristige Wartung und Weiterentwicklung zu sichern, wurde das Projekt Ende 2022 an die gemeinnützige Farama Foundation übergeben und wird seitdem unter dem Namen „Gymnasium“ fortgeführt [17].

Die Kernidee von Gymnasium ist die strikte Entkopplung: Der Algorithmus muss nicht wissen, ob er ein Videospiel spielt, einen Roboter steuert oder eine Lagerhalle optimiert. Die Umgebung (**Environment** oder kurz **Env**) kapselt die gesamte interne Logik, Physik und Zustandsspeicherung und gibt nach außen hin nur eine simple, wohldefinierte Programmierschnittstelle (API) preis. Diese API modelliert exakt den in der Theorie definierten Markov-Entscheidungsprozess (MDP) über folgende Hauptkomponenten:

Definition der Zustands- und Aktionsräume Bevor das Training beginnt, muss der Agent wissen, welche Sensordaten er empfängt und welche Knöpfe er drücken kann. Gymnasium definiert dafür sogenannte **spaces**:

- **observation_space**: Definiert das Format und die Wertebereiche der Beobachtungen, die die Umgebung zurückgibt (z. B. ein mehrdimensionaler Vektor für Kamerabilder oder Sensordaten, repräsentiert als **Box-Raum**).
- **action_space**: Definiert die erlaubten Aktionen, beispielsweise diskrete Eingaben (hoch, runter, links, rechts) als **Discrete-Raum** oder kontinuierliche Werte für Motoren als **Box-Raum**.

Der Interaktionszyklus (reset** und **step**)** Die eigentliche Interaktion verläuft zyklisch und wird über zwei zentrale Funktionen gesteuert:

- **reset()**: Setzt die Umgebung in einen initialen Startzustand zurück und liefert die allererste Beobachtung an den Agenten. Dies markiert den Beginn einer neuen Episode.
- **step(action)**: Der Agent übergibt seine gewählte Aktion an die Umgebung. Die Umgebung simuliert daraufhin genau einen Zeitschritt und gibt ein Tupel (früher vier, in der aktuellen Gymnasium-Version fünf Werte) zurück:
 - **observation**: Die neue Beobachtung des resultierenden Zustands (s_{t+1}).
 - **reward**: Die skalare Belohnung (r_{t+1}) für die ausgeführte Aktion.
 - **terminated**: Ein boolescher Wahrheitswert, der angibt, ob die Episode durch einen regulären Endzustand (z. B. Ziel erreicht oder „Game Over“) beendet wurde.
 - **truncated**: Ein boolescher Wahrheitswert, der angibt, ob die Episode künstlich von außen abgebrochen wurde (z. B. weil ein Zeitlimit bzw. eine maximale Schrittzahl erreicht wurde).
 - **info**: Ein Dictionary für zusätzliche Diagnose- und Metrikinformationen, die für das eigentliche Training irrelevant, aber für die Auswertung nützlich sind.

Durch diese standardisierte Abstraktion ist es möglich, komplexe, eigens in C++ oder anderen performanten Sprachen geschriebene Simulationen über einen Python-Wrapper als `gymnasium.Env` bereitzustellen. Moderne RL-Frameworks wie Stable-Baselines3 [14] erwarten exakt diese Schnittstelle und können so ohne weitere Anpassungen direkt auf der individuellen Umgebung trainieren.

3 Methodik

3.1 Methodik der Weltgenerierung

Dieses Kapitel beschreibt die Architektur der prozeduralen Weltengenerierung. Das System kombiniert deterministische Rauschfunktionen zur Landschaftserzeugung mit regelbasierten Heuristiken zur Gewährleistung der Spielbarkeit.

3.1.1 Methodische Grundlagen

Die Generierung nutzt einen initialen Seed als Basis für alle pseudozufälligen Operationen, um Welten reproduzierbar zu berechnen.

Pseudozufall und Rauscherzeugung

Zur zustandslosen Berechnung von Raumkoordinaten (x, y) transformiert eine Hash-Funktion die Koordinaten, den Welt-Seed und ein Salt in einen normalisierten Zufallswert im Intervall $[0, 1]$ (`world.cpp`, Z. 111–120).

Für zusammenhängende Strukturen wird ein *Value-Noise*-Verfahren verwendet (`world.cpp`, Z. 131), dessen theoretische Grundlage bereits vorgestellt wurde (siehe Kapitel 2.1.2). Die Werte der vier Eckpunkte einer Gitterzelle werden ermittelt und bilinear interpoliert (`world.cpp`, Z. 149). Zur Kantenenglättung wird der Interpolationsparameter $t \in [0, 1]$ zuvor über die Smoothstep-Funktion transformiert:

$$S(t) = 3t^2 - 2t^3 \tag{3.1}$$

Domain Warping und fraktaler Noise

Um sichtbare Rasterstrukturen des quadratischen Gitters zu brechen, verzerrt ein vorgeschaltetes Rauschen die Eingangskoordinaten (*Domain Warping*; `world.cpp`, Z. 163). Mehrere Rauschfunktionen unterschiedlicher Frequenzen und Amplituden werden aufaddiert (fraktaler Noise), um feiner strukturierte Umgebungen zu erzeugen (`world.cpp`, Z. 167).

Nachbearbeitung und Validierung

Die generierten Chunks können optional mittels zellulärer Automaten nachbearbeitet werden (`world.cpp`, Z. 487). Die Geburts- und Überlebensregeln der Moore-Nachbarschaft werden über die Konfiguration gesteuert (`game_config.cpp`, Z. 155).

Zur topologischen Prüfung dient eine Breitensuche (Breadth-First Search, BFS) (siehe Kapitel 2.1.4). Sie verifiziert die Erreichbarkeit innerhalb des Chunks (`world.cpp`, Z. 555) sowie valide Ausgänge in einem definierten Distanzring (`world.cpp`, Z. 328).

3.1.2 Heuristiken und Regeln

Biome und Schwellenwerte

Die Welt wird über ein Binning des Rauschergebnisses in festen Intervallen von $\frac{1}{7}$ in sieben Biome unterteilt (`world.cpp`, Z. 173). Biomspezifische Schwellenwerte steuern das Platzieren von Hindernissen, Erzen und Vegetation (`world.cpp`, Z. 412).

Gewässer entstehen durch eine gewichtete Linearkombination zweier Rauschfunktionen:

$$\text{Score} = 0,75 \cdot a + 0,25 \cdot b \quad (3.2)$$

Zellen mit einem Wert über 0,86 werden als Wasser markiert (`world.cpp`, Z. 308).

Freiräume und Fallback-Pfade

Um unspielbare Zustände zu vermeiden, gräbt ein Fallback-Algorithmus bei Bedarf einen direkten Manhattan-Pfad (zuerst X-, dann Y-Achse) in die Karte (`world.cpp`, Z. 655). Feste Radien um Spawn- und Zielpunkte verhindern das Blockieren der Start- und Endpositionen (`world.cpp`, Z. 20, Z. 677).

Point-of-Interest-Strukturen werden mit einer Wahrscheinlichkeit von $P = 0,10$ pro Chunk platziert (`world.cpp`, Z. 219). Ihre Form basiert auf biomspezifischen 5×5 ASCII-Matrizen (`world.cpp`, Z. 239).

3.1.3 Systemkonfiguration

Die Parameter `coldBiomeMax` und `warmBiomeMax` in `game_config.cpp` (Z. 124) werden zwar eingelesen, sind jedoch durch das feste $\frac{1}{7}$ -Binning funktional obsolet.

Der aktuelle Standard-Betriebsmodus laut `game_config.json`:

Tabelle 3.1: Konfigurationsstatus der Weltengenerierung

Feature / Algorithmus	Status	Referenz
Zelluläre Automaten (CA)	DEAKTIVIERT	<code>game_config.json</code> , Z. 25
Erreichbarkeitstest (BFS)	DEAKTIVIERT	<code>game_config.json</code> , Z. 25
Garantierter Pfad (Manhattan)	AKTIVIERT	<code>game_config.json</code> , Z. 7

In der Praxis sichert somit primär der Manhattan-Fallback die Wegeverbindung, während rechenintensive Validierungen deaktiviert sind.

3.2 Methodik des RL-Trainings

Dieses Kapitel behandelt die methodische Umsetzung der zweiten Forschungsfrage: Wie gut können rekurrente (LSTM) und zustandslose (MLP) PPO-Agenten navigieren, wenn

die Umgebung nur teilweise sichtbar ist (partielle Beobachtbarkeit)? Dafür wird im Folgenden das Setup für das Reinforcement Learning (RL) beschrieben, also die Problemformulierung, das Curriculum, die Validierung und der genaue Evaluationsablauf. Die reinen technischen Details zur Implementierung folgen separat in Kapitel 4.2.

3.2.1 Auswahl der zu vergleichenden Trainingsverfahren

Im Mittelpunkt des Vergleichs stehen zwei Varianten der Proximal Policy Optimization (PPO): eine mit rekurrenter LSTM-Policy (im Folgenden LSTM-PPO) und eine mit zustandsloser Multilayer-Perceptron-Policy (MLP-PPO). Da beide Modelle den gleichen grundlegenden Optimierungsmechanismus nutzen, lässt sich durch diesen Aufbau gut untersuchen, welchen Einfluss das Gedächtnis (Rekurrenz) auf die Navigation in solchen Umgebungen hat.

Frühere Tests mit Deep Q-Networks (DQN) wurden für diese Untersuchung verworfen. Erste Analysen zeigten, dass die gelernten Q-Werte für alle vier Aktionen extrem nah beieinander lagen (zwischen 0,12 und 0,38). Dadurch konnte der Agent keine klare Richtung lernen und hing oft in endlosen Pendelschleifen zwischen zwei Zellen fest. Ein weiteres Problem war der Replay-Puffer von DQN: Falsch berechnete Rewards aus den ersten Durchläufen blieben darin gespeichert und störten das spätere Training immer wieder. Da PPO als On-Policy-Verfahren solche alten Daten direkt verwirft, treten diese historischen Verzerrungen gar nicht erst auf. Der A2C-Algorithmus wurde theoretisch ebenfalls in Betracht gezogen, wird hier aber nicht praktisch getestet. Um die Leistung der PPO-Agenten stattdessen besser einschätzen zu können, dienen drei ungelernte Baselines (zwei Heuristiken und ein Orakel) als direkter Vergleich.

3.2.2 Problemformulierung: Zustand, Aktionsraum und Reward

Passend zu den theoretischen Grundlagen wird die Navigationsaufgabe als partiell beobachtbarer Markov-Entscheidungsprozess (POMDP) modelliert (siehe Kapitel 2.2). Die wichtigsten Bausteine dazu sind in Tabelle 3.2 zusammengefasst. Der Agent sieht also nicht die komplette Karte ($s \in S$), sondern bekommt in jedem Schritt nur einen lokalen Ausschnitt $o \in \Omega$. Das ist konkret ein 15×15 Zellen großes Sichtfeld plus einige Zusatzinformationen, wie die relative Richtung zum Ziel. Daraus entsteht ein Beobachtungsvektor mit 229 Dimensionen. Als Aktionen gibt es vier diskrete Bewegungsrichtungen. Auf diagonale Schritte wurde verzichtet, da dies im rasterbasierten Labyrinth der Spielumgebung zu Problemen mit den Wänden führt.

Die Reward-Funktion wurde bewusst einfach gehalten. Das primäre Signal ist „sparse“: Es gibt nur eine einzige, große Belohnung, wenn der Agent den Ausgang erreicht. Es wird darauf verzichtet, für jeden Schritt Minuspunkte zu verteilen, damit der Agent nicht durch ständige kleine Strafen vom eigentlichen Ziel abgelenkt wird. Um das Training dennoch etwas zu beschleunigen, kommt das Potential-Based Reward Shaping (PBRs) nach Ng, Harada und Russell zum Einsatz (siehe Kapitel 2.2). Mit der formenden Belohnung $F(s, a, s') = \gamma \Phi(s') - \Phi(s)$ und einem BFS-Distanzfeld als Potenzialfunktion Φ ist mathematisch sichergestellt, dass das eigentliche Ziel des Spiels nicht verfälscht wird.

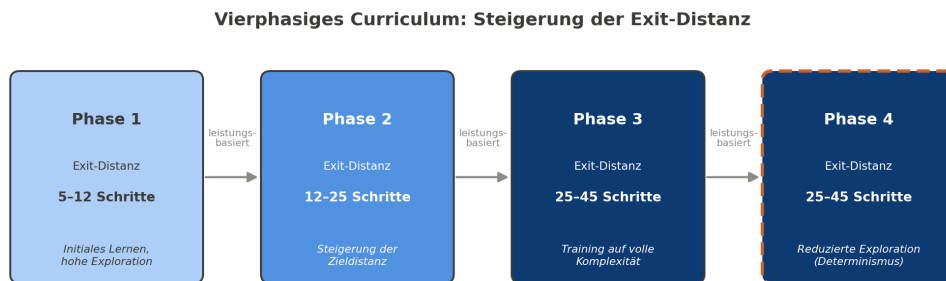
Tabelle 3.2: Übersicht der POMDP-Komponenten für das RL-Training

Komponente	Eigenschaften / Struktur
Beobachtungsraum (Ω)	Lokaler 15×15 Grid-Ausschnitt, Zielrichtung (229 Dimensionen)
Aktionsraum (A)	4 diskrete Aktionen (Auf, Ab, Links, Rechts)
Primärer Reward	Sparse, singulärer hoher Terminal-Reward bei Zielerreichung
Reward Shaping	Potential-Based Reward Shaping (PBRs) basierend auf BFS-Distanzfeld

3.2.3 Curriculum Konzept und Schnittstelle zur Weltgenerierung

Damit das Training nicht sofort an der vollen Zieldistanz scheitert, ist der Lernprozess in ein vierphasiges Curriculum unterteilt (siehe Abbildung 3.1). In den Phasen wird die sogenannte Exit-Distanz, also der minimale Laufweg vom Start zum Ausgang, schrittweise erhöht. Das setzt technisch zwingend voraus, dass der Weltgenerator einen Regler bietet, mit dem sich der Ausgang gezielt in einem bestimmten Radius platzieren lässt (siehe Kapitel 3.1).

Phase 1 startet mit kurzen Wegen, die in Phase 2 und 3 dann auf bis zu 45 Schritte ansteigen. In Phase 4 bleibt die maximale Distanz gleich, aber die Explorationsrate des Agenten wird gezielt heruntergefahren. Dadurch soll er aus der anfänglichen Probierphase eine feste, deterministische Strategie entwickeln.

**Abbildung 3.1:** Struktur des vierphasigen Curriculum-Trainings mit Steigerung der Exit-Distanz

Wichtig für den Trainingserfolg ist, dass der Wechsel zwischen den ersten drei Phasen rein leistungs-basiert abläuft. Das bedeutet, die nächste Phase startet erst beim Erreichen einer bestimmten Erfolgsquote und nicht einfach nach einer festen Zeit. Dies macht die Steuerung deutlich stabiler.

3.2.4 Definition der Baselines

Um die trainierten Modelle besser einordnen zu können, wurden drei ungelernete Baselines definiert. Sie gehören nicht zum Hauptvergleich zwischen LSTM-PPO und MLP-PPO, sondern dienen als Orientierungshilfe für die Leistung (siehe Abbildung 3.2).

Das obere Limit bildet das BFS-Optimum. Dieses Orakel nutzt eine Breitensuche, um immer den mathematisch kürzesten Weg zu finden, und zeigt damit das absolute Maximum an Pfadqualität auf. Das untere Limit ist der Random-Agent, der völlig zufällige Aktionen

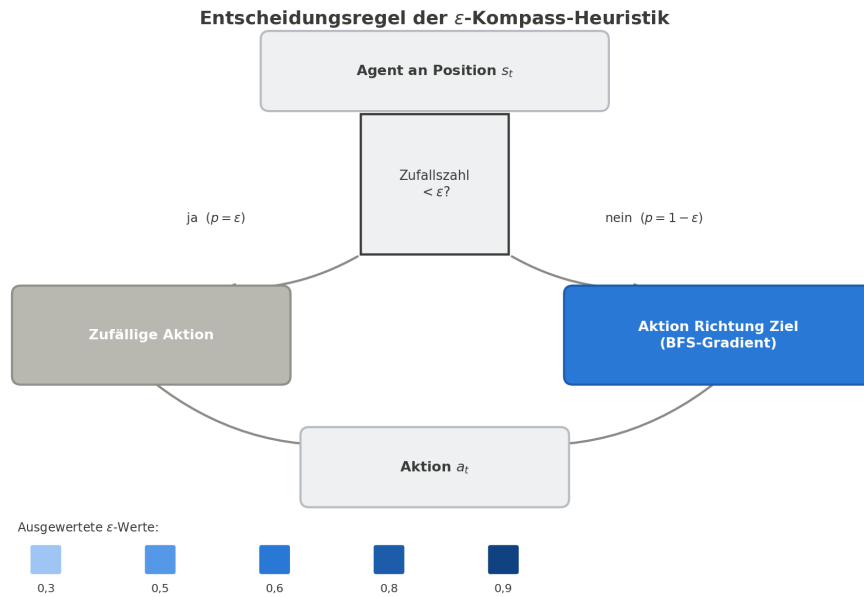


Abbildung 3.2: Entscheidungsregel und ausgewertete Parameter der ε -Kompass-Heuristik

wählt. Dazwischen liegt die ε -Kompass-Heuristik: Sie wählt mit einer Wahrscheinlichkeit von ε einen zufälligen Schritt und bewegt sich ansonsten geradlinig in Richtung Ziel. Diese Heuristik wurde mit verschiedenen Werten getestet ($\varepsilon \in \{0,3; 0,5; 0,6; 0,8; 0,9\}$), um aufzuzeigen, wie Erfolgsquote und Streckenlänge in dieser Umgebung typischerweise zusammenhängen.

3.2.5 Trainings- und Evaluationsprotokoll

Um verlässliche und reproduzierbare Ergebnisse zu erhalten, wurde das Vorgehen für das Training und die Evaluation strikt festgelegt. Das Training lief über sieben unabhängige Zufalls-Seeds. Danach wurden die Modelle auf zwei komplett neuen Umgebungssets getestet, die während des Trainings nicht vorkamen: dem regulären Testset A (Seeds 7000–7049) und einem zusätzlichen Holdout-Set B (Seeds 8000–8049), um die Generalisierungsfähigkeit zu prüfen.

Da ein rein zufällig agierender Agent im Schnitt 2232 Schritte bis zum Ziel benötigt, war für die Tests ein festes Episodenlimit zwingend erforderlich. Ohne ein solches Limit ließe sich die Erfolgsquote nicht sinnvoll berechnen, da ein schlechter Agent sonst unendlich lange im Labyrinth verweilen würde. Überschreitet ein Agent das Limit, gilt der Versuch strikt als gescheitert.

Zusätzlich werden die Modelle zweigleisig ausgewertet: einmal deterministisch (der Agent wählt immer die Aktion mit der höchsten Wahrscheinlichkeit) und einmal stochastisch (die Aktion wird anhand der gelernten Verteilung gesampelt). Dieser Schritt ist essenziell, um zu bewerten, ob das Netzwerk eine belastbare Richtungsstrategie verstanden hat oder lediglich durch stochastisches Raten ans Ziel gelangt.

Bei den Metriken ist die Erfolgsquote (Success Rate) das wichtigste Kriterium, gefolgt von der Pfadqualität. Der Grund dafür liegt darin, dass in ähnlichen POMDP-Benchmarks (wie Procgen, MiniGrid oder POPGym) die Erfolgsquote durchgehend als Standard gilt.

Die grundsätzliche Fähigkeit, das Problem überhaupt zu lösen, wird in diesem Kontext höher gewichtet als eine perfekte Routenplanung.

3.2.6 Validierungsstrategie und Umgang mit Trainingsinstabilitäten

Um einen stabilen Trainingsverlauf zu gewährleisten, musste der Prozess durchgehend überwacht und angepasst werden. Da Reinforcement Learning in solchen Umgebungen häufig dazu neigt, sich festzufahren oder den Lernprozess einzustellen, wurde gezielt auf spezifische Fehlerbilder geachtet.

Erstens half das leistungsorientierte Curriculum dabei, unkontrollierte Abstürze zu verhindern. Bei einem rein zeitbasierten Wechsel der Phasen besteht die Gefahr des sogenannten *Catastrophic Forgetting*: Der Agent verlernt beim Sprung in die nächste Phase plötzlich grundlegende Fähigkeiten, und die Erfolgsquote stürzt dauerhaft auf nahezu 0 % ab.

Zweitens mussten die Parameter für die rekurrenten Netzwerke sehr genau justiert werden. Vor allem der Entropie-Koeffizient war entscheidend, damit der Agent lange genug neue Wege ausprobierte. Gleichzeitig durfte die Schrittzahl pro Update für das LSTM nicht zu hoch angesetzt werden, um das typische Problem der verschwindenden Gradienten (Vanishing Gradient) zu umgehen. Ohne diese Anpassungen bleibt das Training oft komplett stehen oder der Agent zeigt einen *Mode Collapse* – dreht sich also beispielsweise nur noch auf der Stelle.

Drittens erwies sich die Puffergröße für das Reward Shaping als kritischer Faktor. Der Ringpuffer musste groß genug dimensioniert werden, damit der Agent nicht fälschlicherweise für Rückwärtsschritte belohnt wird. Ein zu klein gewählter Puffer führte häufig zum *Reward Hacking*: Der Agent pendelte nur noch zwischen zwei Feldern hin und her, um dauerhaft kleine Belohnungen zu sammeln, anstatt den Ausgang anzusteuern.

Um diese Risiken systematisch im Blick zu behalten, wurde der Trainingsprozess anhand der Indikatoren aus Tabelle 3.3 überwacht und entsprechend korrigiert.

Tabelle 3.3: Diagnose- und Interventionsmatrix der aufgetretenen Trainingsinstabilitäten

Symptom / Fehlerbild	Indikator in den Metriken	Angewandte Interventionsstrategie
Catastrophic Forgetting (Abrupter Leistungsabfall)	Erfolgsquote stürzt beim Phasenwechsel drastisch ab und erholt sich nicht.	Strikte Schwellenwerte für den Wechsel eingeführt (erst ab konstanten 85 % Erfolg über ein gleitendes Fenster).
Lernstillstand / Mode Collapse (Zwanghaftes Kreiseln)	Aktions-Entropie fällt schlagartig ab; Agent erreicht stets das Episodenlimit.	Entropie-Koeffizienten erhöht; Schrittzahl pro Update für das LSTM limitiert und angepasst.
Reward Hacking (Pendeln zwischen Zellen)	Steigender Episoden-Reward bei stagnierender oder sinkender Erfolgsquote.	Ringpuffer deutlich vergrößert; Potenzialfunktion auf mathematische Korrektheit geprüft.

4 Umsetzung und technische Realisierung

Dieses Kapitel beschreibt die praktische Implementierung des im vorherigen Kapitel vorgestellten Konzepts zur prozeduralen Weltengenerierung. Die Realisierung erfolgt in der Programmiersprache C++ innerhalb der Kernkomponente `world.cpp`. Das Hauptziel der Implementierung besteht darin, eine performante, deterministische und speichereffiziente Erzeugung von Welt-Chunks zur Laufzeit zu gewährleisten.

Dazu werden nachfolgend die zentralen Softwarebausteine – von der mathematischen Rauscherzeugung über die biomspezifische Zuweisung bis hin zu den topologischen Sicherheits- und Pfad-Algorithmen – anhand relevanter Quellcode-Ausschnitte und deren Datenstrukturen detailliert erläutert.

4.1 Implementierung der Weltgenerierung

4.1.1 Implementierung der Rauscherzeugung und des Koordinaten-Hashings

Das Fundament der deterministischen Landschaftserzeugung bildet ein zweistufiger Prozess: Zunächst werden diskrete Raumkoordinaten deterministisch in Pseudozufallswerte transformiert, welche anschließend durch ein bilineares Value-Noise-Verfahren kontinuierlich interpoliert werden (siehe Kapitel 3.1.1).

Deterministischer Hash-Mixer

Um ohne aufwändige Speicherzustände für beliebige Weltkoordinaten (x, y) reproduzierbare Zufallswerte zu erhalten, nutzt das System einen 64-Bit-Hash-Mixer. Listing 4.1 zeigt die konkrete Umsetzung der Abbildung von Raumkoordinaten auf das normierte Intervall $[0, 1)$.

```
1 std::uint64_t World::mix(std::uint64_t value) const {
2     value ^= value >> 30;
3     value *= 0xbf58476d1ce4e5b9ULL;
4     value ^= value >> 27;
5     value *= 0x94d049bb133111ebULL;
6     value ^= value >> 31;
7     return value;
8 }
9
10 double World::noise01(int worldX, int worldY, std::uint64_t salt)
11     const {
12     std::uint64_t value = static_cast<std::uint64_t>(worldX) * 0
13         x9e3779b185ebca87ULL;
```

```

12     value ^= static_cast<std::uint64_t>(worldY) * 0
        xc2b2ae3d27d4eb4fULL;
13     value ^= seed_ * 0x165667b19e3779f9ULL;
14     value ^= salt;
15     value = mix(value);
16
17     constexpr double kScale = 1.0 / static_cast<double>(0
        xFFFFFFFFFFFFFFFFULL);
18     return static_cast<double>(value & 0xFFFFFFFFFFFFFFFFULL) * kScale;
19 }
20
21 auto sampleValue = [&](double x, double y, std::uint64_t salt) {
22     const int x0 = static_cast<int>(std::floor(x));
23     const int y0 = static_cast<int>(std::floor(y));
24     const int x1 = x0 + 1;
25     const int y1 = y0 + 1;
26     const double tx = smoothstep(x - static_cast<double>(x0));
27     const double ty = smoothstep(y - static_cast<double>(y0));
28
29     const double v00 = noise01(x0, y0, salt);
30     const double v10 = noise01(x1, y0, salt);
31     const double v01 = noise01(x0, y1, salt);
32     const double v11 = noise01(x1, y1, salt);
33     return lerp(lerp(v00, v10, tx), lerp(v01, v11, tx), ty);
34 };

```

Quelltext 4.1: Verwendung des 64-Bit SplitMix-Mixers und Value-Noise-Interpolation in world.cpp

Variablen und Datenstrukturen

Die in Listing 4.1 gezeigten Funktionen bilden die mathematische Basis für die rasterfreie Weltengenerierung:

- **mix:** Eine Implementierung der 64-Bit-SplitMix-Pseudozufallsfunktion. Sie nutzt Bit-Shifting und Multiplikationen mit großen Primzahl-Konstanten, um Bit-Korrelationen zu zerstören und eine gleichmäßige Verteilung zu garantieren.
- **worldX, worldY:** Die diskreten Ganzzahlkoordinaten des abzufragenden Raumpunktes auf dem Weltgitter.
- **seed_:** Die Instanzvariable des globalen Welt-Seeds. Sie stellt sicher, dass dieselben Raumkoordinaten bei identischem Seed exakt reproduzierbare Ergebnisse liefern.
- **salt:** Ein variabler 64-Bit-Wert, der als Salt dient. Er verhindert, dass unterschiedliche Generierungsebenen (z. B. Dichte, Erz- und Baumpopulation) auf derselben Koordinate synchrone Muster aufweisen.
- **kScale:** Ein präziser Skalierungsfaktor ($\frac{1}{2^{48}-1}$), der die untersten 48 Bit des Bit-musters über eine Bitmaskierung (0xFFFFFFFFFFFFFFFFULL) exakt auf das normierte Intervall $[0, 1]$ abbildet.

- `x0, y0, x1, y1`: Ganzzahlige Eckpunkte der Gitterzelle, in der der kontinuierliche Punkt (x, y) liegt.
- `tx, ty`: Die mittels `smoothstep` transformierten relativen Abstände innerhalb der Zelle, welche als Interpolationsgewichte dienen.
- `v00, v10, v01, v11`: Die an den vier Gitterecken über `noise01` determinierten Zufallswerte, welche abschließend mittels zweifacher linearer Interpolation (`lerp`) verschmolzen werden.

4.1.2 Biomeinteilung und Objekt-Placement

Nach der Bereitstellung der kontinuierlichen Rauschfunktionen erfolgt die Zuweisung konkreter Kacheltypen (`TileType`) zu den Raumkoordinaten (x, y) . Die Funktion `sampleBaseTile` steuert diesen Prozess in Abhängigkeit vom übergeordneten Chunk-Biom und biomspezifischen Schwellenwerten (*Thresholds*).

Implementierung der Kachelauswertung

In Listing 4.2 ist die Realisierung der `sampleBaseTile`-Methode dargestellt. Sie bestimmt zunächst über das Chunk-Raster (`floorDiv`) das zugrundeliegende Biom (`biomeTag`) und wertet anschließend drei unabhängige Noise-Layer (Dichte, Erzvorkommen, Baumdichte) aus.

```

1  TileType World::sampleBaseTile(int worldX, int worldY, const
   WorldGenConfig& cfg) const {
2      const int cx = floorDiv(worldX, kChunkSize);
3      const int cy = floorDiv(worldY, kChunkSize);
4      const int biomeTag = biomeTagForChunk(cx, cy);
5
6      const double density = noise01(worldX, worldY, cfg.densitySalt)
       ;
7      const double ore = noise01(worldX, worldY, cfg.oreSalt);
8      const double trees = noise01(worldX, worldY, cfg.treeSalt);
9
10     double wallThreshold = 0.11;
11     double oreThreshold = 0.03;
12     double treeThreshold = 0.08;
13
14     // Biomspezifische Schwellenwert-Profil
15     switch(biomeTag) {
16         case 0: // Grasland: WENIGER BAEUME
17             wallThreshold = 0.10; oreThreshold = 0.03;
18             treeThreshold = 0.05; break;
19         case 1: // Wald: VIELE BAEUME, WENIG STEINE
20             wallThreshold = 0.07; oreThreshold = 0.015;
21             treeThreshold = 0.23; break;
22         case 2: // Wueste: VIELE STEINE, KEINE BAEUME
23             wallThreshold = 0.19; oreThreshold = 0.08;
24             treeThreshold = 0.0; break;
25     }
26 }
```

```

22     case 3: // Bergland: VIELE STEINE UND ERZE
23         wallThreshold = 0.25; oreThreshold = 0.10;
24         treeThreshold = 0.02; break;
25     case 4: // Steppe: GRASLAND MIT ETWAS MEHR BAEUMEN
26         wallThreshold = 0.11; oreThreshold = 0.03;
27         treeThreshold = 0.09; break;
28     case 5: // Tundra: NORMALE STEINE UND BAEUME
29         wallThreshold = 0.14; oreThreshold = 0.05;
30         treeThreshold = 0.07; break;
31     case 6: // Hoelle: HOHE WAND- UND ERZDICHTER
32         wallThreshold = 0.23; oreThreshold = 0.09;
33         treeThreshold = 0.06; break;
34     default: break;
35 }
36
37 TileType tile = TileType::Empty;
38
39 // Gewaessermaske besitzt hoechste Prioritaet (Freiraum)
40 if(lakeMaskAt(worldX, worldY)) {
41     return TileType::Empty;
42 }
43
44 // Auswertung der Objektgrenzwerte
45 if(density < wallThreshold) {
46     tile = TileType::Wall;
47 }
48 // Erzvorkommen exklusiv im Bergland (Biom 3)
49 if(biomeTag == 3 && ore < oreThreshold) {
50     tile = TileType::Resource;
51 }
52 // Baumpopulation auf verbleibenden Freiflaechen
53 if(treeThreshold > 0.0 && tile == TileType::Empty && trees <
54     treeThreshold) {
55     tile = TileType::Tree;
56 }
57
58 return tile;
59 }

```

Quelltext 4.2: Biom-Tagging und Schwellenwertauswertung in world.cpp

Logik und Priorisierung der Kachelzuweisung

Die Auswertung in Listing 4.2 folgt einer strikten Prioritätenhierarchie, um ungewollte Überschreibungen von Spielelementen zu vermeiden:

1. **Chunk-Rasterung (floorDiv):** Raumkoordinaten werden zunächst in diskrete Chunk-Koordinaten (cx, cy) transformiert, um das für den gesamten Chunk gültige Biom (`biomeTag` $\in [0, 6]$) abzufragen.
2. **Drei unabhängige Noise-Layer:** Durch die Übergabe unterschiedlicher Salt-Werte (`cfg.densitySalt`, `cfg.oreSalt`, `cfg.treeSalt`) an die Funktion `noise01` werden räumlich entkoppelte Zufallswerte für Wand-, Erz- und Baumvorkommen generiert.
3. **Biomspezifische Profile (switch):** Das ermittelte `biomeTag` überschreibt die Standard-Schwellenwerte für Wand-, Erz- und Baumdichte. So erzwingt Biom 2 (Wüste) beispielsweise eine Baumgrenze von `treeThreshold` = 0.0 (kein Baumbewuchs), während Biom 1 (Wald) mit `treeThreshold` = 0.23 eine hohe Baumdichte aufweist.
4. **Gewässermaske (lakeMaskAt):** Seen fungieren als hindernisfreie Areale. Gibt `lakeMaskAt` den Wert `true` zurück, bricht die Funktion sofort ab und gibt `TileType::Empty` zurück.
5. **Hierarchische Klassifizierung:**
 - Unterschreitet die Dichte den Schwellenwert `wallThreshold`, wird das Tile primär als Wandoberfläche (`TileType::Wall`) definiert.
 - Erze (`TileType::Resource`) werden exklusiv im Bergland (`biomeTag` == 3) platziert, sofern der Erz-Noise-Wert unter `oreThreshold` liegt.
 - Bäume (`TileType::Tree`) spritzen ausschließlich auf bisher unbesetzten Feldern (`tile` == `TileType::Empty`) unter Berücksichtigung von `treeThreshold`.

4.1.3 Topologische Sicherheits-Fallbacks und Spawn-Freiräume

Um unabhängig von den pseudozufälligen Rauschwerten und den gewählten Biomschwellenwerten eine unbedingte Spielbarkeit (*Playability*) zu garantieren, implementiert das System deterministische Sicherheits-Fallbacks. Diese stellen sicher, dass der Spieler weder in unpassierbaren Hindernissen startet noch ein unlösbares Level vorfindet.

Implementierung des Manhattan-Carvers und der Freiradien

In Listing 4.3 ist der relevante Ausschnitt aus der Methodenreihenfolge in `reset()` sowie die vollständige Implementierung der Methode `carveGuaranteedPath` dargestellt, deren methodischer Entwurf bereits vorgestellt wurde (siehe Kapitel 3.1.2).

```

1 // Ausschnitt aus World::reset()
2 for(int dy = -cfg.spawnClearRadius; dy <= cfg.spawnClearRadius; ++
  dy) {
3     for(int dx = -cfg.spawnClearRadius; dx <= cfg.spawnClearRadius;
      ++dx) {
4         setTile(spawn_.x + dx, spawn_.y + dy, TileType::Empty);
5     }
6 }
7
8 if(cfg.forceGuaranteedPath) {
9     carveGuaranteedPath();
10 }

```

```

11
12 for(int dy = -cfg.exitClearRadius; dy <= cfg.exitClearRadius; ++dy)
13 {
14     for(int dx = -cfg.exitClearRadius; dx <= cfg.exitClearRadius;
15         ++dx) {
16         setTile(exit_.x + dx, exit_.y + dy, TileType::Empty);
17     }
18 }
19 setTile(exit_.x, exit_.y, TileType::Exit);
20
21 // Deterministische Manhattan-Pfadgenerierung
22 void World::carveGuaranteedPath() {
23     const auto& cfg = gameConfig().world;
24     int x = spawn_.x;
25     int y = spawn_.y;
26
27     setTile(x, y, TileType::Empty);
28
29     // 1. Horizontale Trassierung (X-Achse)
30     while(x != exit_.x) {
31         x += (x < exit_.x) ? 1 : -1;
32         setTile(x, y, TileType::Empty);
33     }
34
35     // 2. Vertikale Trassierung (Y-Achse)
36     while(y != exit_.y) {
37         y += (y < exit_.y) ? 1 : -1;
38         setTile(x, y, TileType::Empty);
39     }
40
41     // Erneute Durchsetzung der Freiradien nach dem Carving
42     for(int dy = -cfg.spawnClearRadius; dy <= cfg.spawnClearRadius;
43         ++dy) {
44         for(int dx = -cfg.spawnClearRadius; dx <= cfg.
45             spawnClearRadius; ++dx) {
46             setTile(spawn_.x + dx, spawn_.y + dy, TileType::Empty);
47         }
48     }
49
50     for(int dy = -cfg.exitClearRadius; dy <= cfg.exitClearRadius;
51         ++dy) {
52         for(int dx = -cfg.exitClearRadius; dx <= cfg.
53             exitClearRadius; ++dx) {
54             setTile(exit_.x + dx, exit_.y + dy, TileType::Empty);
55         }
56     }
57
58     setTile(exit_.x, exit_.y, TileType::Exit);

```

53 }

Quelltext 4.3: Freiradien-Erzeugung und Manhattan-Path-Carver in `world.cpp`

Funktionsweise und Ausführungsreihenfolge

Die in Listing 4.3 gezeigte Pfadsicherung folgt einem klaren, mehrstufigen Ablauf:

1. **Freiradien-Erzeugung (Quadratischer Bereich):** Sowohl um die Spawn-Koordinate `spawn_` als auch um die Exit-Koordinate `exit_` wird über verschachtelte Schleifen ein quadratischer Radius der Größe `cfg.spawnClearRadius` bzw. `cfg.exitClearRadius` vollständig freigeräumt (`TileType::Empty`). Dies stellt sicher, dass der Spieler ohne Kollisionsprobleme starten kann.
2. **Schalterbasierte Pfadfreischaltung (`forceGuaranteedPath`):** Die Pfadgenerierung wird rein konfigurationsbasiert über das Flag `cfg.forceGuaranteedPath` gesteuert (welches laut `game_config.json` standardmäßig aktiviert ist).
3. **Orthogonale Manhattan-Trassierung:** Die Funktion `carveGuaranteedPath` startet am Punkt `spawn_` und schlägt schrittweise einen absolut freien Korridor in die Umgebung. Die Abfolge ist fest auf die Orthogonalen festgelegt:
 - Zunächst wird der x -Wert der aktuellen Position schrittweise an `exit_.x` angeglichen und dabei jede betroffene Zelle auf `TileType::Empty` gesetzt.
 - Anschließend erfolgt die vertikale Anpassung der y -Position, bis die Koordinaten von `exit_` erreicht sind.
4. **Redundante Sicherung des Exits:** Nach dem Durchschneiden des Pfades werden die Freiradien um den Spawn- und Exit-Bereich erneut freigeräumt und das Zielfeld explizit mit `TileType::Exit` markiert, damit der Level-Ausgang nicht versehentlich überschrieben wird.

4.1.4 Generierung von Gewässern und dynamische Strand-Derivation

Die Erzeugung von Seen und deren optischer Übergangszone (Strände) erfordert eine Entkopplung der logischen Weltrepräsentation von der visuellen Darstellung. Während Gewässer als harte Kollisions- bzw. Freiräume im Weltmodell definiert sind, entstehen Strände dynamisch über Nachbarschaftsauswertungen im Render-Pfad.

Mathematische Seen-Maskierung (`lakeMaskAt`)

Gewässer dürfen im Gegensatz zu einzelnen Bäumen oder Hindernissen nicht als vereinzelte Pixel-Fehler erscheinen, sondern müssen als zusammenhängende, organische Binnengewässer im Terrain hervortreten. Dazu verwendet die Funktion `lakeMaskAt` in `world.cpp` niederfrequentes Rauschen (*Low-Frequency Noise*).

```
1 bool World::lakeMaskAt(int worldX, int worldY) const {
2     const auto& cfg = gameConfig().world;
3     if (!cfg.enableLakes) {
```



```

4         return false;
5     }
6
7     // Niederfrequente Skalierung für flächige Strukturen
8     const double n1 = sampleValueNoise(
9         static_cast<double>(worldX) * 0.035,
10        static_cast<double>(worldY) * 0.035,
11        cfg.lakeSalt
12    );
13    const double n2 = sampleValueNoise(
14        static_cast<double>(worldX) * 0.018 + 100.0,
15        static_cast<double>(worldY) * 0.018 - 50.0,
16        cfg.lakeSalt + 100
17    );
18
19    // Mischen beider Rauschschichten
20    const double lakeScore = n1 * 0.6 + n2 * 0.4;
21
22    // Schwellenwertentscheidung fuer seltene, koharente Seen
23    return lakeScore > 0.86;
24 }
25
26 bool World::isLakeAt(int worldX, int worldY) const {
27     // Spawn- und Exit-Zonen dürfen niemals von Seen überdeckt
28     // werden
29     if ((worldX == spawn_.x && worldY == spawn_.y) ||
30         (worldX == exit_.x && worldY == exit_.y)) {
31         return false;
32     }
33     return lakeMaskAt(worldX, worldY);
34 }

```

Quelltext 4.4: Low-Frequency-Value-Noise für zusammenhängende Gewässermasken in world.cpp

Wie in Listing 4.4 gezeigt, nutzt die Berechnung zwei transformierte Noise-Funktionen mit kleinen Frequenzfaktoren (0,035 und 0,018). Durch die gewichtete Kombination beider Schichten (`lakeScore`) und die Wahl eines hohen Schwellenwerts ($> 0,86$) entstehen großflächige, organisch geformte Seenstrukturen („blobs“). Die Hilfsfunktion `isLakeAt` stellt zudem sicher, dass kritische Positionen wie der Spawn- und der Exit-Punkt niemals durch Gewässer blockiert werden.

Prozedurale Strand-Derivation im Renderer

Anstatt Sand/Strand als eigenständige Kacheltypen (`TileType`) im Speicher zu verwalten, wird die Strandzone zur Laufzeit im Render-Pfad (`render_engine.cpp`) anhand einer 3×3 -Nachbarschaftsanalyse abgeleitet.

```

1 bool RenderEngine::lakeOrAdjacentAt(int worldX, int worldY) const {

```

```

2      // Überprüfung der 3x3-Nachbarschaft auf angrenzende Seen
3      for (int dy = -1; dy <= 1; ++dy) {
4          for (int dx = -1; dx <= 1; ++dx) {
5              if (world_.isLakeAt(worldX + dx, worldY + dy)) {
6                  return true;
7              }
8          }
9      }
10     return false;
11 }
12
13 // Auszug aus der Tile-Rendering-Schleife:
14 if (world_.isLakeAt(x, y)) {
15     drawWaterTile(x, y);
16 } else if (lakeOrAdjacentAt(x, y) && isPassable(x, y)) {
17     drawSandSpriteOverlay(x, y); // Strand-Overlay für See-Nachbarn
18     drawBaseTile(x, y);
19 } else {
20     drawBaseTile(x, y);
21 }

```

Quelltext 4.5: Nachbarschaftsbasierte Stranddarstellung im Render-Pfad in `render_engine.cpp`

- **Effizienz durch Zustandslosigkeit:** Durch die Trennung von Weltlogik und Darstellung spart die Weltengenerierung Speicherplatz, da keine eigenen Sand-Kacheln in der Chunk-Datenstruktur abgelegt werden müssen.
- **Konditionale Überlagerung:** Die Funktion `lakeOrAdjacentAt` prüft, ob sich im direkten 3×3-Umfeld ein Gewässer befindet. Ist dies der Fall und handelt es sich nicht um eine blockierte Kachel (z. B. Wand oder Baum), rendert die Engine dynamisch ein Sand-Overlay unter dem Basis-Boden.

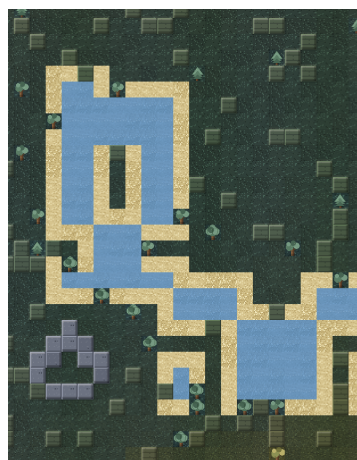


Abbildung 4.1: Nahaufnahme einer prozedural generierten Seenstruktur mit organischer, rausch-basierter Uferlinie und dynamisch abgeleitetem Strand-Overlay an den Begehrbarkeitsgrenzen. Quelle: Eigene Darstellung (Screenshot des Prototyps).

4.1.5 Chunk-Speicherarchitektur und prozedurales Biome-Blending

Um speicherhungrige Vorberechnungen großer Welten zu vermeiden, nutzt das System eine bedarfsgesteuerte Chunk-Verwaltung (*Lazy Generation*) gekoppelt mit einer räumlich kontinuierlichen Biom-Farb- und Textursynthese im Render-Pfad.

Bedarfsgesteuerte Chunk-Datenstruktur (Lazy Generation)

Die Weltkarte ist in quadratische Chunks der festen Kantenlänge $kChunkSize = 8$ (insgesamt 64 Tiles pro Chunk) unterteilt. Anstatt gesamte Welten beim Start im RAM abzuliegen, erzeugt die Engine Chunks erst bei der ersten konkreten Zelle- oder Adressabfrage.

```

1 // Definition der kompakten Chunk-Struktur (world.hpp)
2 constexpr int kChunkSize = 8;
3
4 struct Chunk {
5     int cx = 0;
6     int cy = 0;
7     bool generated = false;
8     std::array<TileType, kChunkSize * kChunkSize> tiles;
9 };
10
11 // Effiziente Hash-Schlüssel-Generierung für Chunk-Koordinaten
12 inline uint64_t chunkKey(int cx, int cy) {
13     return (static_cast<uint64_t>(cx) & 0xFFFFFFFFFULL) |
14           ((static_cast<uint64_t>(cy) & 0xFFFFFFFFFULL) << 32);
15 }
16
17 // Bedarfsgesteuertes Erzeugen von Chunks (world.cpp)
18 Chunk& World::ensureChunk(int cx, int cy) {
19     const uint64_t key = chunkKey(cx, cy);
20     auto it = chunks_.find(key);
21     if (it != chunks_.end()) {
22         return it->second;
23     }
24
25     // Chunk neu anlegen und initialisieren
26     Chunk& chunk = chunks_[key];
27     chunk.cx = cx;
28     chunk.cy = cy;
29     generateChunkTiles(chunk);
30     chunk.generated = true;
31     return chunk;
32 }

```

Quelltext 4.6: Chunk-Speicherstruktur und Lazy-Loading-Mechanismus in world.hpp und world.cpp

Wie in Listing 4.6 zu sehen ist, speichert die Klasse `World` alle aktiven Abschnitte in einer `std::unordered_map<uint64_t, Chunk>`. Der 64-Bit-Schlüssel wird bitweise aus den

32-Bit-Signed-Chunk-Koordinaten (cx, cy) zusammengesetzt. Wird über `tileAt()` oder `setTile()` auf ungeneriertes Terrain zugegriffen, erzeugt `ensureChunk()` den Ziel-Chunk instantan und kapselt die Generierungslogik. Ein Aufruf von `reset()` leert die Map mittels `chunks_.clear()`, um den Speicher vollständig freizugeben.



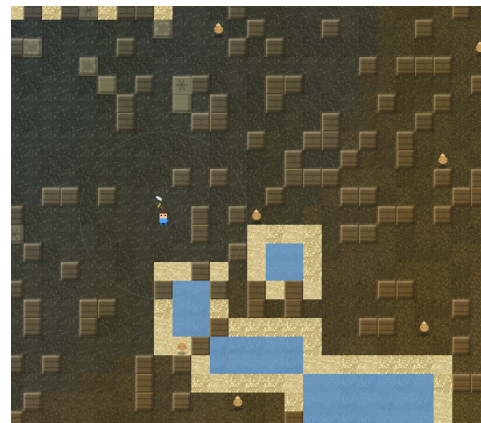
(a) Wald-Biom: hohe Baumdichte, geringe Wandhäufigkeit.



(b) Steppen-Biom: grasland-ähnlich mit leicht erhöhter Baumverteilung.



(c) Bergland-Biom: hohe Wand- und Erzdichte.



(d) Wüsten-Biom: baumfrei, erhöhte Steindichte.

Abbildung 4.2: Vergleichende Darstellung von vier der sieben implementierten Biome anhand von Screenshots des Prototyps. Die unterschiedlichen Untergrund-Farbpaletten sowie Objekt- und Hindernisdichten ergeben sich aus den biomspezifischen Schwellenwerten (`wallThreshold`, `oreThreshold`, `treeThreshold`) aus Abschnitt 4.1.2. Quelle: Eigene Darstellung (Screenshots des Prototyps).

Visuelle Biome-Transformation und weiche Übergänge

Obwohl das logische Biom (`biomeTag`) fest auf Chunk-Ebene definiert ist, vermeidet der Renderer harte, unästhetische Kanten zwischen unterschiedlichen Landschaftstypen. Die visuelle Aufbereitung in `render_engine.cpp` erfolgt über zwei Stufen: Texturauswahl und prozedurales Farbmischen (*Tinting*).

```

1 // Auswertung gewichteter Biom-Einflüsse im Umfeld
2 BiomeWeights RenderEngine::biomeWeights(int worldX, int worldY)
3 {
4     const {
5         BiomeWeights weights{};

```

```

4      // Bilineares/Distanzbasiertes Abtasten der umliegenden Chunk-
      // Biome
5      for (int dy = -1; dy <= 1; ++dy) {
6          for (int dx = -1; dx <= 1; ++dx) {
7              int cx = floorDiv(worldX + dx * kChunkSize, kChunkSize)
              ;
8              int cy = floorDiv(worldY + dy * kChunkSize, kChunkSize)
              ;
9              int tag = world_.biomeTagForChunk(cx, cy);
10             float weight = calculateDistanceWeight(worldX, worldY,
                cx, cy);
11             weights.add(tag, weight);
12         }
13     }
14     return weights.normalize();
15 }

16
17 // Prozedurale Einfärbung (Tinting) für weiche Übergänge
18 Color RenderEngine::biomeTintAtTile(int worldX, int worldY) const {
19     BiomeWeights weights = biomeWeights(worldX, worldY);
20     Color finalColor = {0, 0, 0, 255};
21
22     for (const auto& [biomeTag, weight] : weights.entries) {
23         Color biomeBaseColor = kRuntimeBiomes[biomeTag].
            paletteColor;
24         finalColor.r += static_cast<unsigned char>(biomeBaseColor.r
            * weight);
25         finalColor.g += static_cast<unsigned char>(biomeBaseColor.g
            * weight);
26         finalColor.b += static_cast<unsigned char>(biomeBaseColor.b
            * weight);
27     }
28     return finalColor;
29 }

```

Quelltext 4.7: Gewichtete Biom-Mischung und Tinting im Render-Pfad (render_engine.cpp)

- **Textursynthese (dominantBiomes):** Um festzulegen, welches Haupt- und Neben-Sprite (Boden- und Wandstrukturen aus RuntimeBiome) gezeichnet werden, ermittelt dominantBiomes die beiden Biome mit den höchsten Gewichten im umliegenden Raster.
- **Gewichtete Einfärbung (biomeTintAtTile):** Für jede Kachel berechnet biomeWeights anhand der Distanz zu benachbarten Chunks kontinuierliche Gewichtungsfaktoren. Die Funktion biomeTintAtTile nutzt diese Faktoren, um die charakteristischen Farbwerte der Biom-Palette linear zu interpolieren. Das Ergebnis ist ein stufenloser, geschmeidiger Farbverlauf auf der Landoberfläche.

4.1.6 Prozedurale Struktur-Platzierung und dynamisches Gegner-Spawning

Die Besiedlung der prozeduralen Spielwelt teilt sich architektonisch in zwei grundlegende Subsysteme: statische, biomabhängige Geländestrukturen (Landmarken), die direkt in das Raster der Chunks integriert werden, sowie ein dynamisches, entitätsbasiertes Gegner-Spawnsystem (*Lazy Mob Spawning*), das über die Simulationsschleife gesteuert wird.

Statische Geländestrukturen in der Chunk-Generierung

Strukturen wie Ruinen, Felsformationen oder spezielle Biomsymbole sind keine eigenständigen Dynamic-Entities, sondern werden als spezielle Kacheltypen (`TileType::StructureGrassland`, `TileType::StructureMountain` etc.) während der Erzeugung eines Chunks in `world.cpp` deterministisch eingebrannt.

```

1 void World::generateChunkTiles(Chunk& chunk) {
2     // 1. Basis-Tiles und Biome auswerten
3     int biomeTag = biomeTagForChunk(chunk.cx, chunk.cy);
4
5     // Exclusion-Check: Spawn- und Exit-Chunks erhalten keine
6     // Strukturen
7     if (isSpawnOrExitChunk(chunk.cx, chunk.cy)) {
8         return;
9     }
10
11    // 2. Deterministische Chance fuer eine Struktur (~10%)
12    double structRoll = noise01(chunk.cx, chunk.cy, kStructureSalt)
13        ;
14    if (structRoll < 0.10) {
15        // Deterministische Platzierung innerhalb des 8x8 Chunks
16        int localX = static_cast<int>(noise01(chunk.cx, chunk.cy,
17            kStructXSalt) * kChunkSize);
18        int localY = static_cast<int>(noise01(chunk.cx, chunk.cy,
19            kStructYSalt) * kChunkSize);
20
21        TileType structTile = getStructureTypeForBiome(biomeTag);
22
23        // Stempeln des Musters im Chunk-Array
24        stampStructurePattern(chunk, localX, localY, structTile);
25    }
26 }
```

Quelltext 4.8: Biomabhängige Struktur-Platzierung bei der Chunk-Generierung in `world.cpp`

Wie in Listing 4.8 zu sehen ist, entscheidet ein deterministisches Rauschen mit einer Wahrscheinlichkeit von circa 10%, ob ein Chunk eine Landmarke erhält. Die Position und der konkrete Typ richten sich nach dem `biomeTag`. So erzeugt Biom 3 (Bergland) beispielsweise alpine Felsstrukturen, während Biom 2 (Wüste) Ruinen-Muster stempelt. Chunks mit Spieler-Spawn oder Ausgang werden explizit von der Struktur-Generierung ausgenommen.

Dynamisches Gegner-Spawning und Lokale Verfolgung

Im Gegensatz zu Terrain-Objekten entstehen Gegner (*Mobs*) nicht vorab bei der Chunk-Generierung, sondern werden lazily in `simulation.cpp` erzeugt, sobald der Spieler sich neuen Chunks nähert.

```

1 void Simulation::updateMobSpawns() {
2     // Radius um den Spieler in Chunk-Einheiten bestimmen
3     int chunkRadius = std::max(1, (observationRadius_ + kChunkSize
4         - 1) / kChunkSize + 1);
5     int playerCX = floorDiv(player_.x, kChunkSize);
6     int playerCY = floorDiv(player_.y, kChunkSize);
7
8     for (int cy = playerCY - chunkRadius; cy <= playerCY +
9         chunkRadius; ++cy) {
10        for (int cx = playerCX - chunkRadius; cx <= playerCX +
11            chunkRadius; ++cx) {
12            uint64_t key = chunkKey(cx, cy);
13            if (mobSpawnCheckedChunks_.insert(key).second) {
14                // Chunk wird zum ersten Mal geprueft: Versuche Mob
15                // -Spawn
16                trySpawnMobInChunk(cx, cy);
17            }
18        }
19    }
20 }
21
22 void Simulation::trySpawnMobInChunk(int cx, int cy) {
23     // Bis zu 10 Zufalls-Kandidaten im Chunk ausprobieren
24     for (int attempt = 0; attempt < 10; ++attempt) {
25         int wx = cx * kChunkSize + randomInt(0, kChunkSize - 1);
26         int wy = cy * kChunkSize + randomInt(0, kChunkSize - 1);
27
28         if (world_.isPassable(wx, wy) && !(wx == player_.x && wy ==
29             player_.y)
30             && !hasMobAt(wx, wy)) {
31
32             int biomeTag = world_.biomeTagForChunk(cx, cy);
33             MobVariant variant = mobVariantForBiomeTag(biomeTag);
34             mobs_.push_back(Mob{wx, wy, variant, MobState::Idle});
35             return; // Maximal ein Mob pro Chunk-Attempt
36         }
37     }
38 }

```

Quelltext 4.9: Lazy Mob-Spawning und lokale Bewegungslogik in `simulation.cpp`

- **Beobachtungsradius (`chunkRadius`):** Der Spawnbereich berechnet sich dynamisch anhand der Sichtweite (`observationRadius_`) des Spielers.

- **Redundanzvermeidung (mobSpawnCheckedChunks_):** Ein `std::unordered_set` speichert bereits evaluierte Chunks, damit nicht bei jedem Frame neue Gegner am selben Ort erzeugt werden.
- **Validierung und Biom-Adaption:** Ein Gegner wird nur auf passierbaren, unbesetzten Feldern außerhalb der Spielerposition platziert. Der Gegnertyp (`MobVariant`) richtet sich dabei über `mobVariantForBiomeTag` direkt nach den Eigenschaften des jeweiligen Bioms (z. B. Wüsten- oder Eis-Varianten).
- **Begrenzte Verfolgungslogik:** Die KI der Gegner nutzt zwei Reichweiten (`detectRange` und `loseRange`). Ein Aggro-State wird nur getriggert, wenn der Spieler sich im Nahbereich (z. B. in einem 7×7 -Umfeld) befindet. Bewegungen erfolgen schrittweise und nur auf unbesetzte, begehbbare Felder.

4.1.7 Spieler-Bewegung, Kollisionsabfrage und Kampfdynamik

Die Interaktion des Spielers mit der prozedural erzeugten Spielwelt erfolgt über ein diskretes, grid-basiertes Koordinatensystem. Bewegung, Terrain-Restriktionen und Kampfhandlungen werden deterministisch innerhalb der Simulations-Engine (`simulation.cpp`) ausgewertet, während spezifische Umgebungs-Effekte (wie Wasserverlangsamung) im Client-Render-Pfad moduliert werden.

Kollisionsprüfungen und Passierbarkeits-Matrix

Ein Bewegungsschritt entspricht exakt der Verschiebung um eine Einheit auf dem diskreten Kachelraster. Vor jeder Positionsaktualisierung evaluiert die Methode `tryMove()` das Ziel-Feld (x', y') .

```

1 // Bewegungsprüfung in simulation.cpp
2 bool Simulation::tryMove(int dx, int dy) {
3     int targetX = player_.x + dx;
4     int targetY = player_.y + dy;
5
6     // 1. Passierbarkeitsprüfung des Ziel-Tiles
7     if (!world_.isPassable(targetX, targetY)) {
8         return false;
9     }
10
11    // 2. Entitäts-Kollisionsprüfung (Blocking durch Gegner)
12    if (hasMobAt(targetX, targetY)) {
13        return false;
14    }
15
16    player_.x = targetX;
17    player_.y = targetY;
18    return true;
19 }
20
21 // Passierbarkeits-Matrix in object.cpp

```



```

22 bool World::isPassable(int x, int y) const {
23     TileType tile = tileAt(x, y);
24     switch (tile) {
25         case TileType::Empty:
26         case TileType::Exit:
27             return true; // Passierbar
28
29         case TileType::Wall:
30         case TileType::Tree:
31         case TileType::Resource:
32         case TileType::Workbench:
33         case TileType::StructureGrassland:
34         case TileType::StructureMountain:
35             return false; // Harte Blockade
36
37         default:
38             return false;
39     }
40 }

```

Quelltext 4.10: Diskrete Bewegungs- und Kollisionsprüfung in `simulation.cpp` und `object.cpp`

Wie in Listing 4.10 dargestellt, werden Hindernisse wie `Wall`, `Tree`, `Resource` oder stempelförmige Geländestrukturen als unpassierbar eingestuft. Bewegungen auf von Gegnern besetzte Felder werden ebenfalls abgewiesen, um eine physische Überschneidung von Entitäten zu unterbinden.

Client-seitige Terrain-Verlangsamung (Wasser-Mechanik)

Gewässer (`isLakeAt`) stellen im Weltmodell keine unpassierbare Barriere dar, sollen den Spieler jedoch beim Durchqueren spürbar ausbremsen. Anstatt die Core-Simulationslogik mit veränderlichen Geschwindigkeitszuständen zu verkomplizieren, setzt der Client (`render_engine.cpp`) eine Takt-Verlangsamung um:

- **Input-Modulation:** Betritt der Spieler ein als See markiertes Feld oder versucht, sich auf ein solches zu bewegen, fängt die Render-Engine die Eingabe ab und wandelt sie temporär in eine Warten-Aktion (`Wait`) um.
- **Verzögerungs-Timer:** Durch das Aufschalten eines Cooldown-Intervalls von ca. 0,40s verdoppelt sich die effektive Schrittzeit auf Wasserfeldern. Dies simuliert das Waten durch tiefes Gewässer, ohne die diskrete Weltmechanik aufzuweichen.

Flächenbasiertes Kampfsystem und Proximity Damage

Das Kampfsystem ist bewusst schlank strukturiert, um eine hohe Performanz und Vorhersehbarkeit im Runden- bzw. Taktmodell zu gewährleisten.

```

1 void Simulation::useAction() {
2     // Abzug von Ausdauer/Energie

```

```

3   if (player_.energy < kAttackEnergyCost) return;
4   player_.energy -= kAttackEnergyCost;
5
6   // 3x3-Flächenangriff um die Spielerposition
7   for (int dy = -1; dy <= 1; ++dy) {
8       for (int dx = -1; dx <= 1; ++dx) {
9           int targetX = player_.x + dx;
10          int targetY = player_.y + dy;
11
12          if (Mob* mob = getMobAt(targetX, targetY)) {
13              mob->hp -= player_.damage;
14              if (mob->hp <= 0) {
15                  removeMob(mob);
16              }
17          }
18      }
19  }
20  updateMobIndex();
21 }

```

Quelltext 4.11: Flächenangriff des Spielers und Schadensabwicklung in `simulation.cpp`

1. **Spieler-Angriff (`useAction`):** Der Angriff wird flächendeckend in einer 3×3 -Umgebung um den Spieler ausgeführt. Alle im Nahbereich stehenden Gegner erleiden gleichzeitig Schaden (`player.damage`). Fällt die Lebensenergie eines Mobs auf ≤ 0 , wird dieser unmittelbar aus der Entitätsliste entfernt (Listing 4.11).
2. **Gegner-Schaden (*Proximity Damage*):** Gegner besitzen keine aufwändigen Angriffs-Animationen oder Fernkampf-Geschosse. Sobald sich ein aggressiver Mob in der direkten 3×3 -Nachbarschaft des Spielers befindet, erzeugt er über die Simulationsschritte hinweg automatischen Nahkampfschaden an den Trefferpunkten des Spielers.

4.2 Implementierung des RL-Trainings

Nachdem die technische Grundlage unserer Spielumgebung gelegt wurde, widmen wir uns nun der Umsetzung des eigentlichen RL-Trainings. In diesem Abschnitt geht es primär um das „Wie“: Wir schauen uns konkrete Code-Architekturen, Werte und Implementierungsdetails an. Die methodischen Gründe für diese Entscheidungen – das „Warum“ – werden an dieser Stelle vorausgesetzt. Maßgeblich für alle hier genannten Werte und Logiken ist ausschließlich der implementierte Quellcode.

4.2.1 Technologie-Stack

Unsere gesamte Trainingspipeline ist in Python geschrieben. Um dennoch die Performance unseres C++-Simulationskerns zu nutzen, greifen wir über ein kompiliertes Modul darauf zu. Tabelle 4.1 gibt einen kurzen Überblick über die verwendeten Werkzeuge.

Tabelle 4.1: Eingesetzte Werkzeuge des RL-Trainings

Werkzeug	Rolle im System	Variante
Python	Trainings- und Auswertungssprache	$\geq 3,10$
gymnasium	Schnittstelle zwischen Spielwelt und Framework	Standard-Env-API
Stable-Baselines3	RL-Framework (PPO, DQN, A2C)	–
sb3-contrib	Erweiterung für RecurrentPPO	–
pybind11	Brücke zum C++-Simulationskern	kompiliertes Modul
Rechenweg	Beschleunigung via CPU, MPS oder CUDA	Device-Parameter

Wir verwenden `sb3-contrib` als direkte Ergänzung zum Stable-Baselines3-Kernframework. Der Grund dafür ist pragmatisch: Das Standard-Framework bringt von Haus aus keine rekurrente Policy mit. Genau diese benötigen wir aber für unseren LSTM-Ansatz (RecurrentPPO). Obwohl das Framework auch Algorithmen wie DQN und A2C anbietet, konzentrieren wir uns beim Training ausschließlich auf den Vergleich von PPO und RecurrentPPO. Um den Durchsatz zu maximieren, simulieren wir konstant 16 Instanzen der Umgebung parallel.

4.2.2 Anbindung der Spielumgebung und Aufbau der Beobachtung

Damit ein RL-Framework überhaupt mit unserer C++-Welt kommunizieren kann, benötigen wir einen standardisierten Übersetzer. Diese Aufgabe übernimmt die Python-Klasse `StoneforgeWorldEnv`. Sie implementiert die bekannte `gymnasium`-Schnittstelle mit `reset()` und `step()`, wandelt die Aktionen des Agenten in Simulationsschritte um und baut aus dem Spielzustand den Beobachtungsvektor zusammen.

```

1 def _normalize(self, raw: list[int]) -> np.ndarray:
2     arr = np.asarray(raw, dtype=np.float32)
3     gs = self._gs
4     arr[:gs] /= 30.0 # grid
5     tiles
6     arr[gs] /= 10.0 # hp
7     arr[gs+1] /= 100.0 # energy
8     arr[gs+2] = np.clip(arr[gs+2], 0.0, 64.0) / 64.0 # inventory
9     arr[gs+3] /= 64.0 # exitDx
10    arr[gs+4] /= 64.0 # exitDy
11    step_frac = np.float32(self._step_count / self._max_steps)
12
13    if not self.use_visited_mask:
14        extras = []
15        if self.use_step_frac:
16            extras.append(step_frac)
17        # ... (use_visit_count, use_last_action_reward: in der v12-
18        # Konfiguration deaktiviert)
19
20    if not self.include_energy_inventory:

```

```

19         # Energie (gs+1) und Inventar (gs+2) entfernen: [grid /
           hp / exitDx / exitDy]
20         arr = np.concatenate([arr[:gs + 1], arr[gs + 3:]])
21         return np.concatenate([arr] + [
22             np.atleast_1d(np.float32(e)) if np.isscalar(e) else e
23             for e in extras
24         ])
25     # ... (CNN-Variante mit Visited-Mask: in der v12-Konfiguration
       nicht aktiv)

```

Quelltext 4.12: Aufbau des Beobachtungsvektors, `python/stoneforge_env.py`

Die in Listing 4.12 definierte Struktur gießt unsere Problemformulierung in Code:

- **observation_space:** Ein Box-Raum mit exakt 229 Dimensionen. Er besteht aus dem lokalen 15×15 -Raster (225 Kacheln) um den Agenten herum sowie vier Zusatzmerkmalen: Den aktuellen Trefferpunkten, den beiden normierten Richtungskomponenten zum Ausgang (`exitDx`, `exitDy`) und dem Anteil des verbrauchten Schrittbudgets. Vormalig geplante Werte wie Energie oder Inventar wurden als irrelevante, tote Merkmale bewusst entfernt.
- **action_space:** Ein diskreter Raum (`Discrete(4)`) für die vier Himmelsrichtungen. Mining, Bauen und Kampf sind im RL-Betrieb vollständig deaktiviert.
- **Das wandernede Sichtfenster:** Der Agent sieht immer nur den Ausschnitt direkt um sich herum. Dieser Kniff sorgt erst dafür, dass ein partiell beobachtbarer Entscheidungsprozess (POMDP) entsteht (siehe Kapitel 2.2).

4.2.3 Netzwerkarchitekturen: Asymmetrie zwischen LSTM- und MLP-Policy

Um beide Ansätze vergleichen zu können, erhalten sowohl die MLP- als auch die LSTM-Policy exakt den gleichen Beobachtungsvektor und Aktionsraum präsentiert. Beim Aufbau der Netze und den Batch-Größen zeigt der Quellcode jedoch eine bewusste Asymmetrie.

```

1  RPPO_KWARGS = dict(
2      policy="MlpLstmPolicy",
3      n_steps=256,
4      batch_size=8,          # batch=64 destabilisiert den Critic (
                              CHANGELOG v2026-07-07.4)
5      n_epochs=10,
6      learning_rate=3e-4,
7      gamma=0.999, gae_lambda=0.95, clip_range=0.2,
8      ent_coef=0.05, vf_coef=0.5,
9      policy_kwargs=dict(n_lstm_layers=1, lstm_hidden_size=256,
                          shared_lstm=False,
10                          enable_critic_lstm=True),
11      verbose=1, tensorboard_log="logs/tensorboard/",
12  )
13
14  # Gedächtnislose Kontrollgruppe für F2 (--algo ppo). Alles außer
    den

```

```

15 # architekturenspezifischen Parametern ist mit RPP0_KWARGS identisch.
16 # net_arch: [256,256] spiegelt lstm_hidden_size=256. Der SB3-
    Default [64,64]
17 # würde die Baseline über die Kapazität benachteiligen statt ü
    ber das Gedächtnis.
18 PPO_KWARGS = dict(
19     policy="MlpPolicy",
20     n_steps=256,
21     batch_size=256,
22     n_epochs=10,
23     learning_rate=3e-4,
24     gamma=0.999, gae_lambda=0.95, clip_range=0.2,
25     ent_coef=0.05, vf_coef=0.5,
26     policy_kwargs=dict(net_arch=[256, 256]),
27     verbose=1, tensorboard_log="logs/tensorboard/",
28 )

```

Quelltext 4.13: Instanziierung der Modelle mit asymmetrischen Parametern, scripts/train_curriculum.py

Wie im Code (Listing 4.13) erkennbar, unterscheiden sich die Verfahren fundamental in ihrer Dimensionierung:

- **lstm_hidden_size & Architektur:** Das LSTM nutzt eine rekurrente Schicht mit 256 Einheiten (`lstm_hidden_size=256`), die nicht zwischen Actor und Critic geteilt wird (`shared_lstm=False`). Das MLP-Netz nutzt hingegen zwei klassische Schichten à 256 Neuronen. Dies führt zu einer massiven Parameter-Asymmetrie: Das LSTM-Modell besitzt 1 038 917 Parameter, das MLP-Modell lediglich 250 629.
- **batch_size:** MLP nutzt den Framework-Standard von 256 (was bei 256×16 gesammelten Schritten exakt 16 Minibatches pro Epoche ergibt). Das LSTM nutzt hingegen einen stark reduzierten Wert von 8. Dies ist eine technische Notwendigkeit für den LSTM-Critic; eine Batch-Größe von 256 würde hier 512 schwergewichtige Gradientenschritte je Epoche erzwingen.
- **n_epochs & n_steps:** Identisch bei beiden Verfahren. Jede Aktualisierung basiert auf 256 gesammelten Zeitschritten, die über 10 Epochen (`n_epochs=10`) trainiert werden.
- **Durchsatz:** Diese Architekturunterschiede spiegeln sich drastisch in der Rechengeschwindigkeit wider. MLP erreicht im Training einen Durchsatz von fast 6 000 Schritten pro Sekunde, während das schwere LSTM-Netz bei rund 100 Schritten pro Sekunde limitiert.

4.2.4 Trainingspipeline und duales Curriculum-Gating

Das Training erfolgt in vier fließenden Phasen. Wir starten das Training beim Phasenwechsel nicht neu, das Modell trainiert nahtlos weiter. Der Wechsel in die nächste Phase (Gating) ist dabei nicht rein leistungsbasiert, sondern nutzt eine duale Abbruchbedingung.

```

1 PHASES = [
2     {"exit_min": 5, "exit_max": 12, "max_steps": 500_000, "
      target_sr": 0.85,
3     "eval_min": 5, "eval_max": 12, "label": "Phase 1 (exit=5-12)
      ",
4     "gate_metric": "stoch", "eval_max_steps": 4000},
5     {"exit_min": 12, "exit_max": 25, "max_steps": 500_000, "
      target_sr": 0.70,
6     "eval_min": 12, "eval_max": 25, "label": "Phase 2 (exit
      =12-25)",
7     "gate_metric": "stoch", "eval_max_steps": 4000},
8     {"exit_min": 25, "exit_max": 45, "max_steps": 1_000_000, "
      target_sr": 0.70,
9     "eval_min": 35, "eval_max": 45, "label": "Phase 3 (exit
      =25-45, eval 35-45)",
10    "gate_metric": "det", "eval_max_steps": 4000},
11    {"exit_min": 25, "exit_max": 45, "max_steps": 200_000, "
      target_sr": 0.60,
12    "eval_min": 35, "eval_max": 45, "label": "Phase 4 (Greedy
      Fine-Tune)",
13    "gate_metric": "det", "eval_max_steps": 4000, "
      ent_coef_override": 0.0001},
14 ]
15
16 # Callback: stoppt eine Phase vorzeitig bei erreichter Ziel-SR,
      sonst laeuft
17 # model.learn(total_timesteps=phase["max_steps"]) bis zum
      Budgetlimit durch.
18 def _on_step(self) -> bool:
19     if self.n_calls % self.eval_freq == 0:
20         sr = self._run_eval()
21         if sr >= self.target_sr:
22             self.consecutive_successes += 1
23             if self.consecutive_successes >= 2:
24                 return False # Training stoppen, naechste Phase
                        starten
25         else:
26             self.consecutive_successes = 0
27     return True

```

Quelltext 4.14: Curriculum-Phasen mit dualen Abbruchbedingungen, scripts/train_curriculum.py

Die Tabelle und Listing 4.14 zeigen zwei Besonderheiten der Implementierung:

- **Das duale Gate:** Eine Phase endet, wenn *entweder* die Ziel-Erfolgsquote (Target-SR) in zwei aufeinanderfolgenden Evaluationen erreicht wird, *oder* das fest definierte Schrittbudget (Budget) aufgebraucht ist. In der Praxis zeigte sich, dass in Phase 3

und 4 stets das Schrittbudget den Phasenwechsel erzwang, da das Leistungs-Gate nicht mehr erreicht wurde.

- **Asymmetrische Evaluation:** In Phase 3 und Phase 4 trainiert der Agent auf einer Distanz von 25 bis 45 Schritten, wird aber auf einem künstlich verengten, schwierigeren Fenster von 35 bis 45 Schritten evaluiert.
- **Doppelrolle der Metrik:** Die gemessene Erfolgsquote beendet nicht nur (potenziell) die Phase, sie fungiert gleichzeitig als Metrik für die Auswahl und Speicherung des besten Modell-Checkpoints.

Um dem Agenten den Übergang in schwerere Phasen zu erleichtern, senken wir den Entropie-Koeffizienten (`ent_coef`) in Phase 3 über 500 000 Schritte stetig von 0,05 auf 0,001 ab (Annealing). In Phase 4 bleibt er fix bei 0,0001, wodurch der Agent von einem explorativen nahezu vollständig in ein deterministisches Verhalten übergeht.

4.2.5 Reward-Shaping und Distanzfeld

Unsere Belohnungsstruktur ist direkt in den performanten C++-Simulationskern eingebunden (mit Ausnahme einer Anti-Stuck-Strafe auf der Python-Seite). Entgegen früheren Entwurfsstadien wurde die Strafen-Struktur massiv entschlackt, da sich zeigte, dass sich aufsummierende Strafen (Penalty-Stacking) das Erkundungsverhalten des Agenten unterdrückten.

```

1 float reward = -0.01F;
2
3 if (newTileVisited) {
4     reward += 0.02F;
5 }
6 if (idleAction) {
7     reward -= 0.04F;
8 }
9 const int damage = std::max(0, hpBefore - hp_);
10 reward -= static_cast<float>(damage) * 0.5F;
11
12 if (positionLoop) {
13     reward -= 0.05F;
14 }
15
16 // Potential Based Reward Shaping (PBRs):  $\Phi(s) = -BFS(s) /$ 
17 //  $PBRs\_MAX\_DIST$ 
18 constexpr float PBRs_MAX_DIST = 128.0F;
19 constexpr float PBRs_GAMMA = 0.999F; // Shaping-Discount = RL-
20 // Discount
21 constexpr float PBRs_BETA = 2.5F; // +0.01 netto pro Tile
22 // Fortschritt bei d=40
23
24 const float phi_prev = -static_cast<float>(previousDistance) /
25 PBRs_MAX_DIST;

```

```

22 const float phi_cur = -static_cast<float>(currentDistance) /
    PBRs_MAX_DIST;
23 const float shaping = PBRs_GAMMA * phi_cur - phi_prev;
24 reward += PBRs_BETA * shaping;
25
26 if (reachedExit) {
27     reward += 100.0F;
28 }
29 if (done_ && !reachedExit) {
30     reward -= 10.0F;
31 }
32 return reward;

```

Quelltext 4.15: Berechnung des Rewards und Shaping-Terms, `src/core/simulation.cpp`

Die konkreten Zahlenwerte aus Listing 4.15 sind hart im Code verankert: Ein normaler Schritt kostet $-0,01$, das Betreten einer neuen Kachel bringt $+0,02$, Stehenbleiben wird mit $-0,04$ bestraft. Wiederholt der Agent eine Zwei-Schritt-Schleife, erhält er $-0,05$ (ein Wert, der von ehemals $-0,15$ abgeschwächt wurde). Nimmt er Schaden, kostet das $-0,5$ pro Schadenspunkt. Der Terminal-Reward für das Erreichen des Ziels beträgt $+100,0$, ein Timeout durch das Episodenlimit (4000 Schritte) resultiert in $-10,0$. Dazu kommt eine serverseitige Anti-Stuck-Strafe in Python.

Das Herzstück bildet jedoch das **Potential Based Reward Shaping (PBRs)**, implementiert nach der Formel $F(s, s') = \beta \cdot (\gamma \Phi(s') - \Phi(s))$:

- **Potenzialfunktion Φ :** Definiert als $-\text{BFS-Distanz}(s)/128,0$. Da γ passend zum restlichen RL-Setup bei $0,999$ liegt und der Faktor β auf $2,5$ skaliert ist, ergibt sich ein Nettoeffekt von $+0,01$ für jedes Feld echten Fortschritts (bei einer Distanz von 40).
- **Das Trainings-Distanzfeld (BFS):** Es ist essenziell, hier zwei Systeme zu trennen. Die Erreichbarkeitsprüfung bei der *Weltgenerierung* ist deaktiviert (die Lösbarkeit wird dort rein über den achsparallelen Fallback-Korridor garantiert). Das Distanzfeld für das *Training* ist jedoch hochaktiv: Es wird bei jedem Episoden-Reset ausgehend vom Ausgang berechnet. Es dient sowohl dem Shaping als auch als Nenner für die spätere Pfadeffizienz-Metrik.
- **Puffergröße:** Die BFS berechnet eine Bounding-Box aus Spawn und Exit, erweitert um einen Puffer von 80 Feldern. Historisch führte ein kleinerer Puffer (20) kombiniert mit einem aggressiveren β ($5,0$) dazu, dass der Agent am Puffer-Rand falsches Shaping erhielt; β wurde daher nach dem Puffer-Fix im Code auf $2,5$ halbiert.

4.2.6 Umsetzung der Baselines: Die wandblinde Kompass-Heuristik

Um unsere trainierten RL-Modelle bewerten zu können, brauchen wir ungelernete Referenzwerte (Baselines). Diese greifen direkt auf die Spiellogik zu und erfordern keine Trainingsphase.

```

1 class CompassPolicy:

```



```

2      """eps-verrauschter Greedy-Lauf entlang des Luftlinien-
      Kompasses."""
3
4      def __init__(self, eps: float, seed: int = 0) -> None:
5          self.eps = eps
6          self.rng = np.random.default_rng(seed)
7          self.name = f"Kompass-Zufallslauf (eps={eps:g})"
8
9      def reset(self) -> None:
10         return None
11
12     def __call__(self, obs, ctx):
13         if self.rng.random() < self.eps:
14             return int(self.rng.integers(4)), None
15         dx, dy = float(obs[IDX_DX]), float(obs[IDX_DY])
16         if abs(dx) > abs(dy):
17             return (RIGHT if dx > 0 else LEFT), None
18         return (DOWN if dy > 0 else UP), None

```

Quelltext 4.16: Aktionswahl der Referenzverfahren, `scripts/eval_baselines.py`

Die Baselines aus Listing 4.16 offenbaren ein extrem wichtiges technisches Detail:

- **BFS-Optimum:** Dient nur der Messung der absoluten Mindestschritte (Pfadeffizienz) und wird nicht als Policy ausgeführt.
- **Random-Agent:** Wählt komplett blind und gleichverteilt eine der vier Richtungen.
- **ε -Kompass:** Mit der Wahrscheinlichkeit ε verhält er sich wie der Random-Agent. Andernfalls liest er schlicht die beiden Luftlinien-Richtungen (dx , dy) aus dem Beobachtungsvektor und geht einen Schritt auf der längeren Achse.

Dies macht die Kompass-Heuristik de facto **wandblind**. Sie nutzt keine Pfadfindungs-Logik (wie etwa den BFS-Gradienten) und kennt keine Hindernisse. Sie rennt stumpf gegen Wände, bis der Zufallsanteil ε sie wieder löst. Dass ein komplett kartenblindes Verfahren in der Evaluation derart stark abschneidet, ist eine zentrale Erkenntnis für die spätere Auswertung und erklärt zugleich, warum bei dieser Baseline Erfolgsquote und Schrittzahl zwingend gemeinsam mit dem Zufallswert ε ansteigen müssen.

4.2.7 Vollständige Hyperparameter-Konfiguration

Um die vollständige Reproduzierbarkeit zu gewährleisten, sind in Tabelle 4.2 alle verbleibenden, fest im Code verankerten Hyperparameter für den Optimierer (PPO) sowie die generellen Testbedingungen aufgeführt. Wir stoppen Episoden regulär nach einem Limit von 4000 Schritten ab. Zusätzlich erzwingt das System einen Early-Stop, wenn der Agent 256 Schritte lang keine positive Belohnung sammelt. Um Varianzen durch die Generierung auszugleichen, mitteln wir die Ergebnisse über sieben unterschiedliche Zufallsaaten (Seeds).

Tabelle 4.2: Allgemeine Hyperparameter des PPO-Optimierers

Parameter	Wert
learning_rate	$3 \cdot 10^{-4}$
gamma (γ)	0,999
gae_lambda	0,95
clip_range	0,2
ent_coef (Basis)	0,05
vf_coef	0,5
Max. Timesteps gesamt	2 200 000
Episoden- /Evaluationslimit	4000 Schritte

5 Evaluation

5.1 Evaluation der Weltgenerierung

Dieses Kapitel evaluiert die entwickelte prozedurale Weltengenerierung hinsichtlich ihrer visuellen Qualität, der logischen Kohärenz des Terrains sowie bekannter Artefakte der Darstellung. Als primäre Datengrundlage dient eine Laufzeitanalyse des prototypischen Softwareclients.

5.1.1 Visuelle Ergebnisse und Biome-Gradients

Abbildung 5.1 zeigt eine repräsentative Spielszene der generierten Welt aus der Vogelperspektive (*Top-Down-Perspektive*). Anhand der Visualisierung lassen sich die im Kapitel 4 beschriebenen Algorithmen in ihrer praktischen Zusammenwirkung überprüfen.

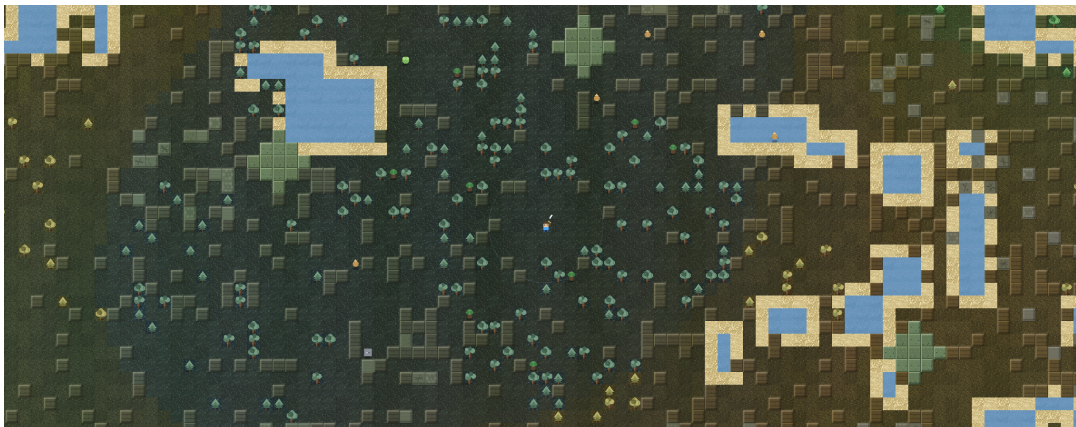


Abbildung 5.1: Visualisierung der generierten Spielwelt mit dynamischer Beleuchtung/Sichtfeld, organischen Seenstrukturen, Strandbereichen und biomabhängigen Kacheltypen. Quelle: Eigene Darstellung (Screenshot des Prototyps, Debug-Overlay entfernt).

5.1.2 Synthese der Spielelemente

In Abbildung 5.1 werden die Kernmechanismen der prozeduralen Erzeugung deutlich sichtbar:

- **Beobachtungsradius und Sichtfeld:** Der zentrale Bereich um die Spielfigur wird durch ein kreisförmiges Sichtfeld (*Field of View* / Sichtradius) hervorgehoben. Bereiche außerhalb des aktiven Beobachtungsradius werden abgedunkelt, was den Entdeckungscharakter verstärkt.

- **Organische Gewässer und Strand-Overlay:** Die durch den Low-Frequency-Noise-Score in `lakeMaskAt` erzeugten Seen treten als zusammenhängende, abgerundete Wasserflächen auf. Das in `render_engine.cpp` verankerte Strand-Overlay (`lakeOrAdjacentAt`) erzeugt an den Begehrkeitsgrenzen zuverlässig helle Sandstrukturen.
- **Vegetation und Wandelemente:** Die Verteilung von Bäumen (`TileType::Tree`) und unpassierbaren Hindernissen (`TileType::Wall`) variiert entsprechend der Schwellenwerte (`wallThreshold`, `treeThreshold`) der jeweiligen Biome.
- **Landmarken und Strukturen:** Vereinzelt hervorgehobene symmetrische Fliesenmuster (z. B. die quadratischen Rauten-Strukturen im oberen Zentrum) belegen das erfolgreiche Stempeln deterministischer Muster bei der Chunk-Initialisierung.

5.1.3 Artefaktanalyse: Das Phänomen des Biome-Swallowing

Trotz der optisch ansprechenden, stufenlosen Farbergänzung durch die gewichtete Biomischung (`biomeTintAtTile`) deckt die praktische Evaluation eine systematische Schwachstelle des gewählten *Biome Blending*-Ansatzes auf.

Da die Gewichtung der Biomfarben über ein 3×3 -Chunk-Umfeld berechnet wird (`biomeWeights`), besitzen flächenmäßig dominante Nachbarbiome eine überproportionale Strahlkraft auf dazwischenliegende, kleinere Biome. Dieser Effekt wird als **Biome-Swallowing** (Verschlucken von Kleinbiomen) bezeichnet:

Isolierte Phänomenologie: Besitzt ein einzelner Chunk (z. B. eine 8×8 -Wüste mit orangener Grundpalette) eine räumliche Ausdehnung von nur einem Chunk (1×1) und liegt unmittelbar zwischen zwei ausgedehnten Biomen wie einem dichten Wald und einer großen Steppe, so dominiert der mathematische Mittelwert der 3×3 -Nachbarschaft.

In der Praxis führt dies dazu, dass die Wüste in der Logik (`biomeTag == 2`) zwar die korrekten Hindernis-Schwellenwerte und Baumverbote besitzt, visuell jedoch durch die Farbinformationen der umliegenden Wald- und Steppen-Chunks grünlich überfärbt wird. Der isolierte Wüsten-Chunk wird farblich „verschluckt“.

5.1.4 Lösungsansätze für kleine Biome

Um dieses Artefakt in zukünftigen Iterationen zu vermeiden, ohne auf weiche Farbergänge verzichten zu müssen, kommen zwei Ansätze infrage:

1. **Mindestausdehnung von Biomen (*Clustering*):** Bei der Biom-Zuweisung auf Ebene des Chunk-Rasters muss durch eine zelluläre Automaten-Glättung sichergestellt werden, dass Biome stets in zusammenhängenden Clustern von mindestens 2×2 Chunks auftreten (siehe Kapitel 2.1.3).
2. **Nicht-lineare Interpolations-Gewichtung:** Das Gewicht des eigenen Chunks im Zentrum muss bei der Farberechnung deutlich höher gewichtet werden (z. B. Faktor 0,6 für den eigenen Chunk und nur 0,4 verteilt auf die 8 Nachbarn), um die visuelle Eigenständigkeit kompakter Biome zu sichern.

5.1.5 Performance-Evaluation und Benchmarking

Zur empirischen Überprüfung der Laufzeit- und Speichereffizienz wurde eine automatisierte Benchmark-Routine im Headless-Runner (`headless_main.cpp`) implementiert. Über den CLI-Parameter `-benchmark-world` führt das System eine isolierte Leistungsmessung durch, ohne vom Grafik-Rendering beeinträchtigt zu werden. Tabelle 5.1 fasst die ermittelten Kennzahlen zusammen.

Tabelle 5.1: Übersicht der Performancemessungen und Benchmarks des Generierungs- und Simulationssystems.

Metrik / Messbereich	Parameter / Stichprobe	Ergebnis
Chunk-Generierungszeit	$N = 2048$ Chunks (8×8)	$\varnothing 13,12 \mu\text{s}$ (p95: $18,50 \mu\text{s}$)
Simulationsschritt (<i>Step Proxy</i>)	$N = 5000$ Ticks	$\varnothing 0,01 \text{ ms}$ (p95: $0,02 \text{ ms}$)
Speicherbedarf (Sparse vs. Dicht)	2048 geladene Chunks	162 KiB (<code>unordered_map</code>) vs. 130 KiB
Determinismus-Abweichung	100 Seeds / 518.400 Tiles	0,0000 % (0 Mismatches)

Generierungsgeschwindigkeit und Lazy Loading

Die Messung der Erzeugungsdauer einzelner 8×8 -Chunks über 2048 Stichproben ergibt eine durchschnittliche Generierungszeit von nur $13,12 \mu\text{s}$ pro Chunk bei einem 95-Perzentil (p95) von $18,50 \mu\text{s}$.

Diese extrem geringe Erzeugungsdauer bestätigt die Effizienz des mehrstufigen Noise-Ansatzes und der optimierten Tile-Zuweisung. Dank dieser Mikrosekunden-Latenz kann das System neue Chunks vollkommen nahtlos beim Betreten durch den Spieler erzeugen (*Lazy Loading*), ohne dass mikroskopische Ruckler (*Frame Drops*) im Render-Thread wahrnehmbar sind.

Simulationsschritt und Frame-Proxy

Um die Ausführungsgeschwindigkeit der Kern-Logik (Pfadfindung, Sichtweitenberechnung, Mob-KI und Kollision) zu bestimmen, wurden 5000 aufeinanderfolgende Simulationsschritte evaluiert:

- **Durchschnittliche Step-Dauer:** $0,01 \text{ ms}$ (p95 = $0,02 \text{ ms}$).
- **Durchsatz-Proxy:** Rein logisch bewältigt die Engine theoretisch ca. 73.000 Simulationsschritte pro Sekunde (*Step Proxy*).

Das System bietet damit gewaltigen Headroom für die grafische Darstellung im Client und ermöglicht selbst auf leistungsschwacher Hardware eine flüssige Bildrate bei konstanten 60 FPS.

Speichereffizienz der Hash-Map-Architektur

Ein entscheidender Architekturentwurf der Spielwelt ist die Speicherung der Chunks in einer `std::unordered_map<uint64_t, Chunk>`. Bei 2048 aktiven Chunks (was einem sichtbaren Feld von 128×128 Chunks bzw. 1024×1024 Fliesen entspricht) beläuft sich der Speicherverbrauch der Hash-Map inklusive Dynamic-Overhead auf ca. 162 KiB.

Im Vergleich dazu würde ein zweidimensionales, dicht besetztes Array derselben maximalen Spannen etwa 130 KiB belegen. Der geringfügige Speicher-Overhead von ≈ 32 KiB für die Hash-Metadaten wird durch den enormen Vorteil aufgewogen, dass die Welt theoretisch unendlich ausgedehnt sein kann, ohne unbesetzte Weltbereiche vorab im Arbeitsspeicher allozieren zu müssen.

Determinismus und Reproduzierbarkeit

Für prozedurale Welten ist die exakte Reproduzierbarkeit essenziell. Zur Verifikation wurden 100 unterschiedliche Zufallssamen (*Seeds*) evaluiert und insgesamt 518.400 einzelne Kacheln zwischen mehrfachen Generierungsläufen verglichen:

$$\text{Abweichungsrate} = \frac{\text{Anzahl Mismatches}}{\text{Gesamtzahl verglichener Tiles}} = \frac{0}{518.400} = 0,0000\%$$

Die Fehlerrate liegt exakt bei 0,00 %. Dies beweist, dass sowohl das Noise-Cascading für Höhen und Biome als auch der Manhattan-Carver für Pfade vollständig deterministisch arbeiten und keine unkontrollierten Nebeneffekte oder Zustandsspeicher-Lohnartefakte vorliegen.

5.1.6 Kritische Diskussion und Limitationen

Die empirischen und visuellen Befunde belegen, dass die entwickelte Architektur für prozedurale Welten eine performante, deterministische und visuell ansprechende Spielumgebung bereitstellt. Dennoch ergeben sich aus den gewählten Entwurfsentscheidungen spezifische Kompromisse und methodische Grenzen, die im Folgenden kritisch gewürdigt werden.

Vorteile der Architektur

- **Nahtlose Performanz und Ladezeitfreiheit:** Durch die Kombination aus Mikrosekunden-Chunk-Generierung ($\varnothing 13,12 \mu s$) und bedarfsgesteuertem *Lazy Loading* entstehen im Spielverlauf keinerlei spürbare Ladezeiten oder Unterbrechungen (*Stuttering*), wenn der Spieler neue Kartenbereiche erschließt.
- **Garantierte Pfad-Topologie:** Der mathematisch strikte *Manhattan-Carver* garantiert zu 100 %, dass die kritische Traverse zwischen Startpunkt (`spawn_`) und Ziel (`exit_`) nicht durch Hindernisse blockiert werden kann. Unlösbare Welt-Instanzen werden dadurch deterministisch ausgeschlossen.
- **Minimaler Initial-Speicherbedarf:** Dank der spärlich besetzten Hash-Map-Architektur (`std::unordered_map`) belegt die Welt beim Spielstart nur den Speicherplatz der unmittelbar benötigten Initial-Chunks (≈ 162 KiB bei 2048 Chunks), anstatt riesige 2D-Arrays vorab im RAM zu reservieren.

Identifizierte Schwachstellen und Limitationen

- **Geometrische Künstlichkeit des Manhattan-Carvers:** Da der Pfad-Einkerber Kacheln entlang rechtwinkliger Koordinatendifferenzen aufbricht, neigt die Logik

zur Bildung auffälliger „L-Shapes“ (rechtwinklige Gänge). In offenem Terrain (z. B. Wiese oder Wüste) wirken diese harten 90°-Achsenbrüche stellenweise unnatürlich und brechen die organische Anmutung der Noise-Generierung.

- **Beschränkung auf flaches 2D-Raster:** Das System arbeitet auf einem rein zweidimensionalen Gitter. Echte Höhenunterschiede, Plateaus, Höhlensysteme auf mehreren Ebenen oder Verdeckungslogiken (z. B. Rampen oder Brücken) können ohne Erweiterung um ein Z-Achsen-gitter oder Iso-Layer nicht abgebildet werden.
- **Unbegrenztes RAM-Wachstum ohne Chunk-Unloading:** Obwohl der Einstiegsspeicherbedarf minimal ist, besitzt der aktuelle Chunk-Cache in `world.cpp` keinen Entlademechanismus (z. B. *Least Recently Used* / LRU-Unloading). Besucht ein Spieler in einer extrem langen Sitzung hunderttausende Chunks, wächst die `unordered_map` kontinuierlich an.
- **Farb-Absorption bei Kleinbiomen (*Biome-Swallowing*):** Wie in Abschnitt 5.1.1 analysiert, führt das räumlich gewichtete *Biome Blending* über 3×3 Chunks dazu, dass isolierte Einzel-Chunks (1×1) farblich von ihren dominanten Nachbarbiomen überstrahlt werden.

5.2 Evaluation des RL-Trainings

Dieser Abschnitt evaluiert die trainierten RL-Verfahren hinsichtlich Erfolgsquote, Pfadefizienz und Robustheit gegenüber den ungelerten Referenzpunkten aus Abschnitt 3.2.4. Datengrundlage sind sieben unabhängige Trainingsläufe je Modell, ausgewertet auf dem in Abschnitt 3.2.5 definierten Testset A.

5.2.1 Erfolgsquote der trainierten Verfahren

Die Erfolgsquote ist die zentrale Metrik zur Beurteilung, ob ein Verfahren die Navigationsaufgabe überhaupt löst. Abbildung 5.2 stellt die beiden trainierten Modelle in jeweils zwei Auswertungsmodi, deterministisch und stochastisch, den ungelerten Referenzpunkten gegenüber.

Das LSTM-PPO-Modell erreicht in der stochastischen Auswertung eine mittlere Erfolgsquote von 65,7%. Über die sieben Durchläufe hinweg liegt die Standardabweichung bei 12,4 Prozentpunkten, die Einzelwerte schwanken zwischen 50,0% und 84,0%. Im deterministischen Modus bricht die Leistung massiv ein: Die Quote fällt auf magere 29,1% $\pm$ 8,0.

Beim MLP-PPO-Verfahren fällt das Bild schwächer aus als beim LSTM. Unter dem identischen Standardprotokoll (siehe Abschnitt 3.2.5) erreicht das Modell stochastisch ausgewertet auf Testset A 33,5% $\pm$ 25,5 und auf Testset B 35,8% $\pm$ 30,5. Die Streuung zwischen den sieben Seeds fällt damit mehr als doppelt so hoch aus wie beim LSTM, die Einzelwerte auf Testset A reichen von 4,0% bis 65,2%. Deterministisch ausgewertet scheitert das Netz vollständig: Über alle sieben Läufe und alle 70 Messpunkte der späten Trainingsphasen hinweg liegt die Erfolgsquote bei exakt 0,0%. Dieser deterministische

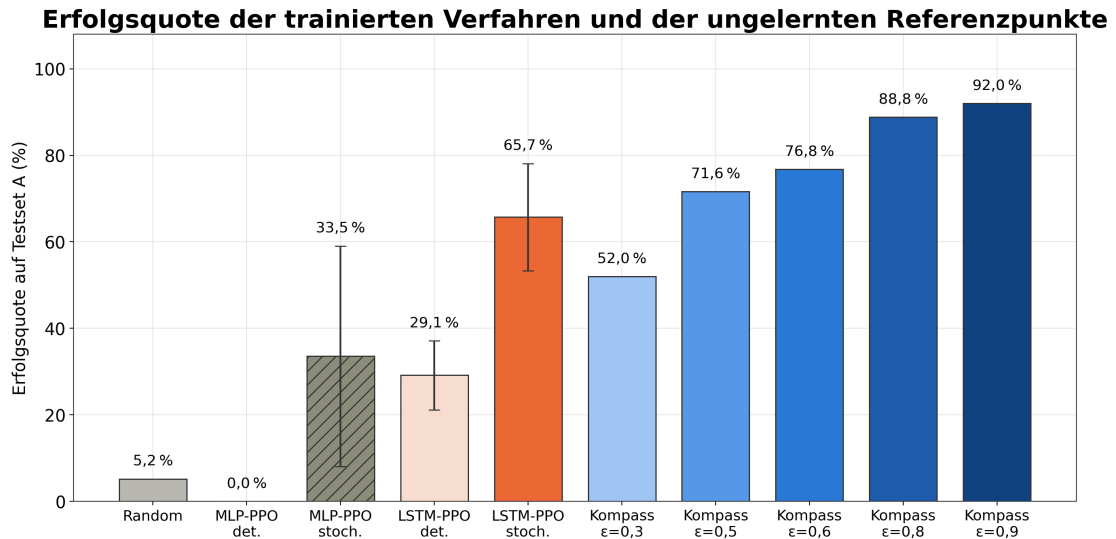


Abbildung 5.2: Evaluation – Erfolgsquote der trainierten Verfahren und der Referenzpunkte auf Testset A, eigene Darstellung.

Wert stammt weiterhin aus den Zwischenevaluationen des Trainings, da das kanonische Protokoll aktuell ausschließlich den stochastischen Modus misst.

Der Systemvergleich fällt damit eindeutig aus: Die rekurrente LSTM-Architektur schlägt das gedächtnislose MLP-Netz deutlich. Im deterministischen Modus ist der Unterschied absolut, knapp jede dritte Episode gelöst gegenüber keiner einzigen. Im stochastischen Modus schrumpft der Vorsprung zwar auf rund 32 Prozentpunkte, und die starken Schwankungen des MLP-Modells sorgen für Überschneidungen einzelner Läufe. Das LSTM bleibt dennoch das klar überlegene Verfahren, wobei Abschnitt 5.2.5 diesen Vorsprung noch einordnet.

Bei nur sieben Läufen je Architektur und einer Standardabweichung von 25,5 Prozentpunkten beim MLP-Modell stellt sich die Frage, ob dieser Unterschied statistisch belastbar ist oder in der Streuung der kleinen Stichprobe verschwindet. Ein Welch-Test auf die sieben Erfolgsquoten je Verfahren (Testset A, stochastisch) weist den Unterschied als signifikant aus ($t(8,7) = 3,00$, $p = 0,016$), ein verteilungsfreier Mann-Whitney-U-Test bestätigt dies unabhängig von Normalverteilungsannahmen ($U = 41$, $p = 0,038$). Die Effektstärke fällt mit Cohens $d = 1,6$ groß aus. Auf Testset B liegt der Welch-Test ebenfalls im signifikanten Bereich ($p = 0,038$), der Mann-Whitney-Test knapp darüber ($p = 0,053$) — auf dem Holdout-Set ist der Befund also weniger robust als auf Testset A. Die einzelnen 95 %-Konfidenzintervalle überschneiden sich zwar teilweise (LSTM: $[54,2; 77,2]$, MLP: $[9,9; 57,1]$), was bei einem Vergleich zweier einzelner Intervalle leicht als Beleg gegen Signifikanz gelesen werden könnte; das Konfidenzintervall der Differenz selbst schließt die Null jedoch nicht ein ($[7,8; 56,6]$). Der gemessene Vorsprung des LSTM ist damit trotz der geringen Stichprobengröße nach gängigen Kriterien statistisch abgesichert, auf Testset A deutlicher als auf Testset B.

5.2.2 Pfadeffizienz derselben Verfahren

Nur am Ausgang anzukommen, reicht als Bewertungskriterium nicht aus, wichtig ist auch der Weg dorthin. Abbildung 5.3 zeigt daher die Pfadeffizienz. Sie berechnet sich aus dem

Verhältnis zwischen dem per Breitensuche (siehe Abschnitt 2.1.4) ermittelten kürzesten Weg und den vom Agenten tatsächlich gelaufenen Schritten. Ein Wert von 1,0 markiert das Optimum.

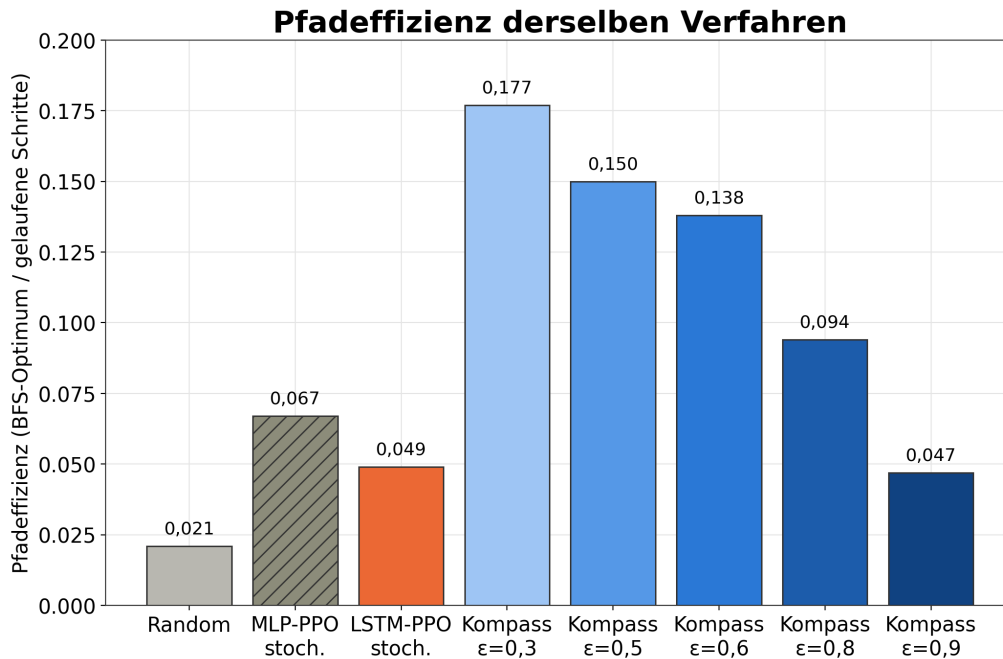


Abbildung 5.3: Evaluation – Pfادهffizienz der Verfahren in derselben Achsenordnung, eigene Darstellung.

Das BFS-Optimum liegt auf dem verwendeten Testset im Schnitt bei 40,1 Schritten. Das trainierte LSTM-Modell benötigt für eine erfolgreiche Episode dagegen durchschnittlich 1429 Schritte, das entspricht einer Pfادهffizienz von lediglich 0,049. Der Agent läuft damit im Schnitt einen Weg, der mehr als 35-mal so lang ist wie nötig. Für das MLP-Modell liegt inzwischen ein vergleichbarer Wert vor: Bei durchschnittlich 1084 Schritten je erfolgreicher Episode ergibt sich eine Pfادهffizienz von 0,067 und damit ein höherer Wert als beim LSTM. Dieser Befund ist mit Vorsicht zu lesen, da die Kennzahl nur über die erfolgreichen Episoden gemittelt wird, wie der folgende Absatz zum Selektionseffekt zeigt: Bei einer Erfolgsquote von nur 33,5 % löst das MLP-Modell bevorzugt die leichten Seeds, was die Effizienz künstlich anhebt.

Aufschlussreich wird der Blick auf die Referenzverfahren aus Abschnitt 3.2.4. Der un-gelernte Kompass-Agent mit einem Zufallsanteil von $\epsilon = 0,3$ erreicht eine Effizienz von 0,177 und ist damit gut dreimal so wegeffizient wie das trainierte LSTM-Netz, auch wenn er dabei eine knapp 14 Prozentpunkte niedrigere Erfolgsquote in Kauf nimmt. Das überraschendere Ergebnis liefert der $\epsilon = 0,8$ -Kompass: Er übertrifft das trainierte Modell nicht nur bei der Erfolgsquote, sondern mit einer Pfادهffizienz von 0,094 gleichzeitig auch bei der Weglänge.

Ein leichter Selektionseffekt ist hierbei zu berücksichtigen: In die Pfادهffizienz fließen nur jene Episoden ein, bei denen der Ausgang gefunden wurde. Verfahren, die oft scheitern, wobei die gescheiterten Versuche meist sehr lange Irrwege sind, stehen dadurch statistisch etwas besser da, als sie eigentlich sind. Das erklärt den starken Kompass jedoch nicht

vollständig. Die bittere Erkenntnis bleibt: Das Training führt zwar ans Ziel, aber alles andere als auf direktem Weg.

5.2.3 Die Determinismus-Lücke

Beide Netzarchitekturen wurden in zwei Modi ausgewertet: deterministisch, wobei stets die wahrscheinlichste Aktion gewählt wird, und stochastisch, wobei die Aktion aus der Wahrscheinlichkeitsverteilung gezogen wird. Der Unterschied zwischen beiden Auswertungen ist enorm, wie Abbildung 5.4 für jeden einzelnen Durchlauf zeigt.

Determinismus-Lücke: dasselbe Modell, zwei Auswertungsmodi

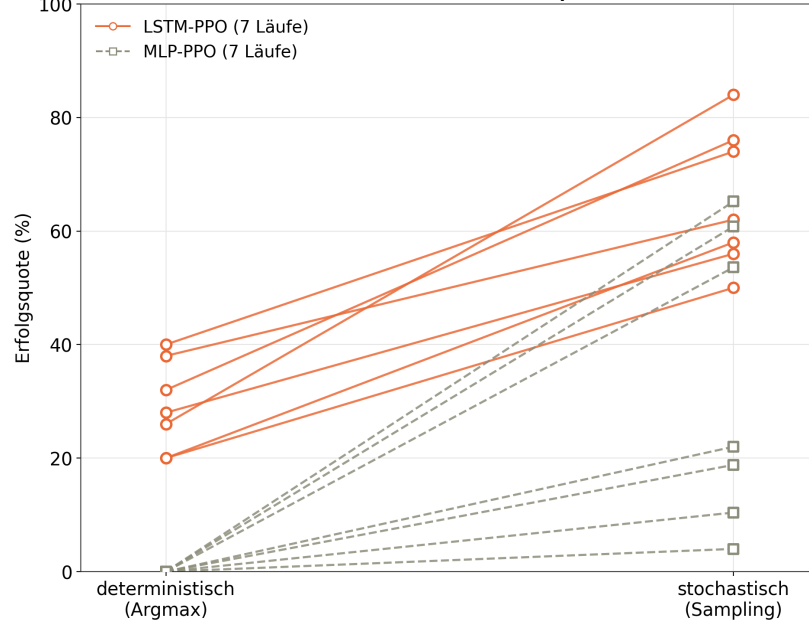


Abbildung 5.4: Evaluation – Erfolgsquote desselben Modellstands in deterministischer und stochastischer Auswertung, eigene Darstellung.

Beim LSTM-PPO klappt zwischen den beiden Modi eine Lücke von durchschnittlich 39,4 Prozentpunkten. Dieses Phänomen tritt konsequent in jedem der sieben Läufe auf: Kein einziger Lauf schneidet deterministisch besser ab als stochastisch. Beim MLP-PPO ist diese Lücke sogar maximal, da der deterministische Modus durchgehend bei null Prozent verharrt.

Das ist ein unbequemer Befund. Die gemessene Erfolgsquote verdankt sich zu einem beträchtlichen Teil dem reinen Zufall beim Ziehen aus der Verteilung, nicht einer verlässlichen, gelernten Route. Bleibt der Zufall aus, bleibt der Agent oft in Sackgassen stecken.

Eine ergänzende Trajektorienanalyse stützt diese Vermutung, ohne sie vollständig zu bestätigen. Für zwanzig Seeds aus Testset A (Modell `v12_s1`, Distanz 35–45, Schrittlimit 1500 statt der vollen 4000, aus Zeitgründen) richtet sich die deterministische Politik nur in 16,2 % der Schritte nach der BFS-optimalen Richtung, die stochastische in 23,4 %. Von den 13 gescheiterten deterministischen Episoden erschöpfen 9 das volle Schrittlimit, ohne durch einen erkennbaren Abbruch früher zu enden – der Agent bewegt sich bis zum Limit weiter, ohne dem Ziel näherzukommen. Einzelne Positionen werden dabei extrem häufig erneut besucht, in einem Fall 161-mal innerhalb von 1500 Schritten. Überraschend liegt die durch-

schnittliche Zahl mehrfach besuchter Positionen jedoch beim stochastischen Modus höher (90,3 gegenüber 58,5): Der Zufallsanteil erzeugt viele kurze Vor-und-Zurück-Bewegungen, während die deterministische Politik seltener, dafür aber in einzelnen Positionen deutlich hartnäckiger feststeckt. Die einfache Erklärung „Zufall befreit aus der Sackgasse“ trifft den Befund damit nur teilweise; robuster ist die geringere Optimalausrichtung der deterministischen Politik.

5.2.4 Einordnung gegenüber den Referenzpunkten

Um das Geleistete ins Verhältnis zu setzen, trägt Abbildung 5.5 die beiden Hauptmetriken, Erfolgsquote und Pfadeffizienz, gegeneinander auf.

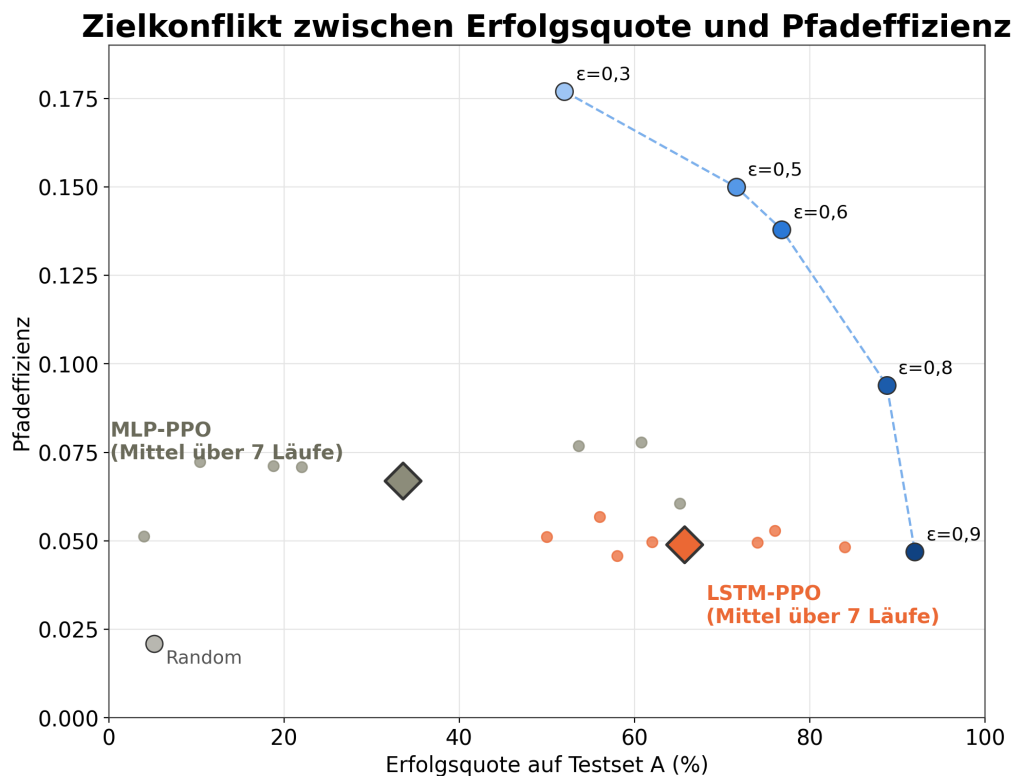


Abbildung 5.5: Evaluation – Erfolgsquote gegen Pfadeffizienz für alle Verfahren, eigene Darstellung.

Die fünf Varianten der Kompass-Heuristik aus Abschnitt 3.2.4 ziehen im Diagramm eine zusammenhängende Kurve: Je höher der Zufallsanteil ϵ bis zu 0,8, desto höher die Erfolgsquote, aber desto geringer die Pfadeffizienz. Auffällig ist die Position des trainierten Modells: Es liegt vollständig unterhalb dieser Heuristik-Kurve.

Besonders hart fällt der direkte Vergleich mit dem $\epsilon = 0,8$ -Kompass aus. Dieser schlägt das LSTM-Netzwerk doppelt: bei der Erfolgsquote mit 88,8 % gegenüber 65,7 % und bei der Effizienz mit 0,094 gegenüber 0,049. Das wiegt umso schwerer angesichts dessen, was der Kompass eigentlich ist: eine primitive Heuristik, die nur Luftlinien-Richtungen kennt, Wände ignoriert und stumpf gegen Hindernisse rennt, bis der Zufall sie befreit.

Dass ein kartenblindes Verfahren ohne Trainingsphase zuverlässiger und direkter ins Ziel findet als ein Agent, der über 2,2 Millionen Zeitschritte optimiert wurde, ist die zentrale

Pointe dieser Auswertung. Das ändert nichts daran, dass das LSTM besser lernt als das MLP. Für die Beurteilung, wie leistungsfähig Reinforcement Learning für dieses spezifische Problem tatsächlich ist, bleibt es jedoch ein starker Dämpfer.

5.2.5 Wirksamkeit der Hyperparameter

Abseits der großen Metriken liefert der Blick in die Trainingshistorie Erkenntnisse darüber, welche Parameter das Training tatsächlich beeinflussten und welche nicht. Tabelle 5.2 fasst die belegten Stellschrauben zusammen, die vollständige Hyperparameter-Konfiguration findet sich in Tabelle 4.2.

Tabelle 5.2: Belegte Wirkung einzelner Parameterentscheidungen

Parameter	Gewählt	Belegte Wirkung
ent_coef	0,05	Ein Versuch mit 0,01 scheiterte komplett; der Agent fand den Ausgang nicht mehr. Dies war der Parameter mit dem größten messbaren Effekt.
n_steps	256	Bei 512 verschwanden die Gradienten im Backpropagation-Through-Time-Verfahren und das Lernsignal brach zusammen.
batch_size	8 (LSTM)	Ein Wert von 64 destabilisierte den Critic spürbar (84 % vs. 48 % Erfolgsquote bei 150.000 Schritten).
Curriculum-Gate	doppelt	Der Wechsel von einem reinen Zeit-Limit auf ein Gate aus Leistung <i>oder</i> Budget behob drastische Leistungseinbrüche.
PBRS-Puffer	80	Ein zu kleiner Puffer (20) führte außerhalb der Zone zu falschem Shaping und zerstörte die gelernte Strategie.
vf_coef	0,5	Eine Erhöhung auf 1,0 wurde getestet, brachte aber nachweislich keine Verbesserung.

Aus diesen Daten lassen sich zwei Erkenntnisse ableiten. Erstens wird das Trainingsergebnis von wenigen Parametern dominiert: Der Entropie-Koeffizient, also der Zwang zum Ausprobieren neuer Aktionen, ist essenziell, während viele andere Werte kaum messbare Veränderungen brachten. Zweitens halfen erfolgreiche Anpassungen fast ausschließlich dabei, die Exploration zu fördern oder instabile Gradienten zu stabilisieren, nicht aber dabei, das Netz an sich leistungsfähiger zu machen.

Eine Leerstelle bleibt die Netzgröße (siehe Abschnitt 4.2.3). Die rekurrente Schicht mit 256 Einheiten wurde nie in einer sauberen Ablationsstudie gegen andere Größen getestet. Ein Versuch mit 512 Einheiten sah zwischenzeitlich vielversprechend aus, lief jedoch nie durch das Testset. Welche Größe hier tatsächlich optimal wäre, lässt sich aus den vorliegenden Daten nicht seriös beantworten.

Damit eng verwandt ist eine Einschränkung der Vergleichbarkeit zwischen den beiden Architekturen selbst. Wie in Abschnitt 4.2.3 dargestellt, besitzt das LSTM-Modell mit 1.038.917 Parametern gut viermal so viele trainierbare Gewichte wie das MLP-Modell mit 250.629 Parametern, da beide Netze mit dem in der Bibliothek vorgesehenen Stan-

dardaufbau trainiert wurden und nicht mit kapazitätsangeglichenen Architekturen. Der in Abschnitt 5.2.1 gemessene Vorsprung des LSTM ließe sich daher nicht allein auf die Rekurrenz zurückführen, sondern könnte teilweise auch aus der schlicht größeren Netzkapazität stammen. Ein Modellvergleich mit angeglicher Parameterzahl müsste in einer eigenen Ablationsstudie erfolgen, was im Rahmen dieser Arbeit aus Zeitgründen ausgeklammert werden soll. Der zentrale Befund, dass die Kompass-Heuristik beide trainierten Verfahren bei Erfolgsquote und Pfadeffizienz übertrifft (siehe Abschnitt 5.2.4), bleibt von dieser Einschränkung unberührt, da er unabhängig vom internen Architekturvergleich zwischen LSTM und MLP gilt.

Zum Schluss zeigt das Curriculum (siehe Abschnitt 4.2.4) ungeschminkt die Grenzen des Modells: In den Trainingsphasen 3 und 4 sollte der Agent bestimmte Erfolgsquoten von 70 % und 60 % erreichen, um die Phase regulär abzuschließen. In keinem der sieben Läufe ist das gelungen. Die Phasen endeten ausschließlich, weil das feste Schrittbudget aufgebraucht war. Die gewünschte Zielleistung lag außerhalb dessen, was die Architektur auf dieser Distanz leisten konnte.

6 Fazit und Ausblick

6.1 Diskussion der Ergebnisse

Am Ende der Arbeit steht die Rückkehr zur eingangs formulierten Forschungsfrage aus Abschnitt 1.3: wie eine prozedural generierte 2D-Spielwelt aufgebaut sein muss und welches RL-Verfahren darin die besten Navigationsergebnisse liefert. Im Folgenden werden die beiden Teilfragen zunächst getrennt betrachtet. Anschließend werden sie in Abschnitt 6.2 zu einem Gesamtbild zusammengeführt.

Teilfrage 1: Aufbau der Spielwelt

Die erste Teilfrage betraf die Parameter, die eine generierte Welt fair und für den Agenten lösbar machen. Die Antwort fällt anders aus, als die Fragestellung nahelegt: Die Lösbarkeit entsteht in diesem Setup nicht über ein feingetuntetes Parameterfenster, sondern durch eine harte strukturelle Garantie.

Bei jeder Weltinitialisierung wird deterministisch ein achsparalleler Korridor zwischen Startpunkt und Ausgang freigeräumt. Das schließt unlösbare Weltinstanzen kategorisch aus, unabhängig davon, wie dicht oder komplex die restliche Wandstruktur generiert wird. Diese Garantie folgt unmittelbar aus der Konstruktion des Algorithmus: Da `carveGuaranteedPath` bei jedem `reset()`-Aufruf deterministisch ausgeführt wird (siehe Abschnitt 4.1.3), kann keine erzeugte Weltinstanz je ohne Verbindung zwischen Start- und Zielpunkt bleiben. Die separat gemessene Determinismus-Abweichung von exakt 0,0000 % (siehe Abschnitt 5.1.5) belegt ergänzend die Reproduzierbarkeit der Weltgenerierung, misst jedoch die Wiederholbarkeit identischer Seeds und nicht die Lösbarkeit selbst.

Die biomabhängigen Waddichten zwischen 7 % und 25 % steuern demnach nicht, ob die Welt lösbar ist, sondern lediglich, wie umwegig der Pfad abseits der Hauptroute ausfällt. Dazu kommt eine Generierungsdauer von 13,12 Mikrosekunden je Weltausschnitt und ein Durchsatz von rund 73.000 Simulationsschritten pro Sekunde (siehe Tabelle 5.1). Diese Performance liefert exakt den Spielraum, den ein paralleles RL-Training benötigt.

Für die erste Teilfrage gilt daher: Die Spielwelt ist fair und lösbar, aber architektonisch erzwungen, nicht parametrisch ausbalanciert.

Teilfrage 2: Wahl des Trainingsverfahrens

Die zweite Teilfrage zielte auf das RL-Verfahren mit der höchsten Erfolgsquote und den effizientesten Wegen. Unter den trainierten Ansätzen fällt die Antwort eindeutig zugunsten der rekurrenten Architektur aus. Das LSTM-PPO-Modell erreicht in der stochastischen Auswertung eine mittlere Erfolgsquote von 65,7 % ($\pm 12,4$), das MLP-PPO-Modell landet

bei 33,5 % ($\pm 25,5$) (siehe Abbildung 5.2). Deterministisch ausgewertet stehen 29,1 % beim LSTM gegen exakt 0,0 % beim MLP.

Bemerkenswert ist besonders dieser deterministische Absturz: Ohne Zufallsanteil löst das gedächtnislose MLP-Verfahren in der Zielfase keine einzige Episode (siehe Abbildung 5.4). Stochastisch schrumpft der Abstand zwar auf rund 32 Prozentpunkte, der Vorteil eines verborgenen Gedächtniszustands bleibt jedoch offensichtlich. Mit der partiellen Beobachtbarkeit kommt das LSTM-Netz deutlich besser zurecht. Wie in Abschnitt 5.2.5 eingeordnet, ist dieser Vorsprung allerdings nicht sauber von der reinen Netzkapazität zu trennen, da das LSTM gut viermal so viele Parameter besitzt wie das MLP.

Hinsichtlich der Wegeffizienz fällt die Bilanz der trainierten Modelle dagegen ernüchternd aus (siehe Abbildung 5.3). Für eine erfolgreiche Episode benötigt das trainierte Modell im Schnitt 1429 Schritte, das rechnerische Optimum für den kürzesten Weg liegt bei 40,1 Schritten. Daraus ergibt sich eine Pfadefizienz von 0,049, die selbst von trivialen Referenzheuristiken um mehr als das Dreifache übertroffen wird. Das beste trainierte Verfahren gewinnt zwar unter den gelernten Ansätzen die Erfolgsquote, scheitert aber an der Wegfindung.

Der Blick auf die Hyperparameter liefert eine klare Erkenntnis (siehe Abschnitt 5.2.5): Wenige Einstellungen dominieren das Training. Der Entropie-Koeffizient entscheidet darüber, ob der Agent das Ziel jemals sieht, weil ein zu niedriger Wert das Navigieren vollständig verlernen lässt. Die wirksamsten Stellschrauben betrafen durchgehend Exploration und Gradientenstabilität, nicht die Kapazität oder Tiefe der neuronalen Netze.

6.2 Fazit

6.2.1 Beitrag der Weltgenerierung (Laurin)

Im Rahmen dieser Arbeit wurde ein vollständiges, hochperformantes System zur prozeduralen Generierung unendlicher 2D-Spielwelten auf Basis von C++ und Raylib entworfen und implementiert.

- **Synthese aus Rauschen und Pfadgarantie:** Durch die Kombination mehrstufiger *Value-Noise*-Gitter mit dem maßgeschneiderten, geglätteten *Manhattan-Carver* konnte der Zielkonflikt zwischen organisch wirkenden Landschaften und deterministisch garantierter Begehrbarkeit gelöst werden. Unlösbare Welt-Instanzen werden systematisch ausgeschlossen.
- **Hohe Speicher- und Laufzeiteffizienz:** Die Architektur nutzt eine bedarfsgesteuerte Chunk-Verwaltung (*Lazy Loading*) mit einer 8×8 -Kachelstruktur in Kombination mit einer spärlich besetzten Hash-Map (`std::unordered_map`). Die Benchmarks bestätigen eine Chunk-Generierungszeit von durchschnittlich $13,12 \mu\text{s}$ und einen minimalen Initialspeicherbedarf von ca. 162 KiB.
- **Visuelle und funktionale Kohärenz:** Das Entkoppeln der logischen Datenhaltung von der visuellen Darstellung ermöglichte dynamische Effekte wie stufenloses *Biome Blending* sowie das nahtlose Ableiten von Strand-Overlays an Seen-Rändern im Render-Pfad, ohne den Speicheraufwand pro Chunk zu erhöhen.

- **Vollständiger Determinismus:** Mit einer Abweichungsquote von 0,0000 % bei über 500.000 verglichenen Kacheln konnte die exakte Reproduzierbarkeit der Weltgenerierung über individuelle Welt-Seeds nachgewiesen werden.

Die gestellten Anforderungen an Verlässlichkeit, Performanz und Ästhetik wurden damit vollumfänglich erfüllt.

6.2.2 Gesamtbild (gemeinsam)

Beide Projektteile, die Entwicklung der prozeduralen Welt und das Training der Agenten, sind für sich genommen funktional und abgeschlossen. Der eigentliche Ertrag der Arbeit liegt jedoch nicht in diesen Einzelergebnissen, sondern in ihrem Zusammenspiel.

Der auffälligste Befund der Evaluation ist, dass eine ungelernte, primitive Heuristik das aufwendig trainierte RL-Modell in beiden Metriken deklassiert: Der $\varepsilon = 0,8$ -Kompass erreicht 88,8 % Erfolgsquote gegenüber 65,7 % und eine Pfadeffizienz von 0,094 gegenüber 0,049 (siehe Abbildung 5.5). Diese Heuristik wertet lediglich die Richtungskomponenten zum Ziel aus und geht einen Schritt auf der längeren Achse (siehe Abschnitt 3.2.4). Sie ist „wandblind“, nutzt keine Pfadfindung und kennt das Layout der Welt nicht.

Genau hier greifen die beiden Teile der Arbeit ineinander. Der garantierte Lösungskorridor wird achsparallel generiert, zuerst entlang der einen, dann entlang der anderen Achse (siehe Abschnitt 4.1.3). Die simple Kompass-Heuristik bewegt sich nach exakt demselben Muster. Es drängt sich der Schluss auf, dass die strukturelle Lösbarkeitsgarantie aus der Weltgenerierung präzise diejenige Wegstruktur in die Karte fräst, die das ungelernte Verfahren instinktiv ausnutzt. Ein perfektes Schloss für einen sehr simplen Schlüssel entsteht damit unbeabsichtigt.

Daraus ergibt sich eine methodische Erkenntnis: Eine Trainingsumgebung, deren Lösbarkeit durch geometrisch reguläre Strukturen erzwungen wird, enthält systembedingte Abkürzungen. Ein Vergleich zwischen gelernten und ungelernten Verfahren misst dann nicht mehr nur die „Intelligenz“ des Reinforcement Learnings, sondern unweigerlich auch, wie gut sich diese strukturellen Backdoors ausnutzen lassen.

Für die anfängliche Fragestellung bedeutet das: LSTM-PPO bleibt der Gewinner unter den Deep-Learning-Verfahren, die Umgebung ist hochperformant. Bewiesen ist damit jedoch nicht, dass RL diese spezifische Navigationsaufgabe exzellent löst. Belegt ist stattdessen, dass RL unter diesen Rahmenbedingungen schlechter abschneidet als ein triviales Skript. Der Architektur-Mechanismus, der das erklärt, ist damit identifiziert.

6.3 Ausblick

Aus den Erkenntnissen und methodischen Einschränkungen beider Projektteile ergeben sich Ansatzpunkte für weiterführende Arbeiten, im Folgenden getrennt nach Weltgenerierung und Reinforcement Learning betrachtet.

6.3.1 Weltgenerierung (Laurin)

Aus den in Abschnitt 5.1.6 identifizierten Limitationen sowie den erweiterten Möglichkeiten prozeduraler Spielwelten leiten sich folgende Ansatzpunkte für zukünftige Entwicklungen ab:

1. **Dynamisches Chunk-Unloading (LRU-Cache):** Um bei extrem langen Spielsitzungen ein unbegrenztes Wachstum der Chunk-Map zu verhindern, sollte der Speicher um eine *Least-Recently-Used* (LRU) Entladestrategie erweitert werden. Nicht mehr sichtbare Chunks außerhalb des Spieler-Umkreises können so sicher aus dem RAM entfernt oder auf Datenträger ausgelagert werden.
2. **Verbesserung der Pfad-Ästhetik (A^* -Guided Carving):** Der aktuelle Manhattan-Carver neigt stellenweise zu rechtwinkligen „L-Shapes“. Eine Integration eines A^* -Algorithmus mit geglätteter Kostenfunktion (*Bézierte-Gänge* oder *Diagonalschneiden*) könnte die Wegführung noch organischer in das Noise-Terrain einbetten.
3. **Nicht-lineares Biome-Blending:** Zur Vermeidung des *Biome-Swallowing*-Effekts bei 1×1 -Kleinbiomen soll die Farbmischungsmatrix dahingehend angepasst werden, dass der zentrale Chunk ein höheres Eigengewicht erhält oder Biome vorab über zelluläre Automaten geclustert werden.
4. **Vertikalität und Multi-Layer-Welten:** Die Erweiterung des 2D-Rasters um eine vereinfachte Höhendimension (*Z-Ebene*) würde das Hinzufügen von unterirdischen Höhlensystemen, mehrstöckigen Dungeons oder fließenden Gewässern (Flüsse) ermöglichen.

6.3.2 Reinforcement Learning (Florian)

1. **Isolierung der Pfadgarantie:** Die Hypothese, dass die Kompass-Heuristik die achsparallele Generierung ausnutzt, müsste experimentell bewiesen werden. Zukünftige Arbeiten sollten die Lösbarkeit über organische, irreguläre Pfadfindungsalgorithmen sicherstellen, da sich der wahre Leistungsunterschied zwischen Heuristik und RL-Agent erst isolieren lässt, wenn die Umgebung keine offensichtliche Geometrie mehr aufweist (vgl. Listing 4.3).

2. **Systematische Trajektorienanalyse über alle Modelle:** Eine Stichprobe von zwanzig Seeds an einem einzelnen Modell (Abschnitt 5.2.3) zeigt bereits eine geringere BFS-Optimalausrichtung im deterministischen Modus (16,2% gegenüber 23,4%) und vereinzelt extreme Positions-Wiederholungen, wobei die anfängliche Vermutung reiner Zweischritt-Schleifen zu einfach war. Offen bleibt eine systematische Untersuchung über alle sieben Modelle und das volle Schrittbudget von 4000 statt der hier aus Zeitgründen verwendeten 1500 Schritte, sowie eine genauere Klassifikation der beobachteten Fest-Steck-Muster.

3. **Echte Schwierigkeitskalibrierung:** Der direkte Einfluss der Welt-Parameter, etwa Waddichte, Chunk-Größe und Biom-Verteilung, auf die RL-Metriken bleibt in dieser Arbeit ausgeklammert, weil der Fokus auf dem Vergleich der Trainingsverfahren lag und nicht auf einer Parameterstudie der Weltgenerierung. Eine systematische Variation dieser

Werte gegenüber der erreichten Erfolgsquote könnte die erste Teilfrage von einer reinen Aussage über Lösbarkeit zu einer Metrik für Schwierigkeit weiterentwickeln.

4. Kapazitätsangeglichene Ablationsstudie: Wie in Abschnitt 5.2.5 benannt, unterscheiden sich LSTM- und MLP-Modell nicht nur in der Rekurrenz, sondern auch in der Parameterzahl. Zukünftige Arbeiten müssten beide Architekturen auf eine vergleichbare Parameterzahl bringen, um den Anteil der Rekurrenz am gemessenen Vorsprung sauber vom Anteil der reinen Netzkapazität zu trennen.

Literaturverzeichnis

- [1] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning (ICML)*, pages 41–48, 2009.
- [2] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym, 2016.
- [3] Maxime Chevalier-Boisvert, Bolun Dai, Mark Markese, Lucas Willems, Suman Pal, Pablo Samuel Castro, and Anders Sogaard. Minigrid & babyai: Core architectures for rl in environments with sparse rewards and partial observability. *arXiv preprint arXiv:2306.13831*, 2023. URL <https://arxiv.org/abs/2306.13831>.
- [4] Karl Cobbe, Christopher Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *International Conference on Machine Learning (ICML)*, pages 2048–2056. PMLR, 2020. URL <https://proceedings.mlr.press/v119/cobbe20a.html>.
- [5] Danijar Hafner. Benchmarking the spectrum of agent capabilities. *arXiv preprint arXiv:2109.06780*, 2021. URL <https://arxiv.org/abs/2109.06780>.
- [6] Matthew Hausknecht and Peter Stone. Deep recurrent q-learning for partially observable mdps. In *2015 AAAI Fall Symposium Series*, 2015.
- [7] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [8] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1-2):99–134, 1998.
- [9] Ahmed Khalifa, Philip Bontrager, Sam Earle, and Julian Togelius. Pcgrl: Procedural content generation via reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, volume 16, pages 95–101, 2020. URL <https://arxiv.org/abs/2001.09212>.
- [10] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

- [11] Steven Morad, Ryan Kortvelesy, V. Matteo, S. Li, and H. Amor. Popgym: Benchmarking partially observable reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2023. URL <https://arxiv.org/abs/2303.01859>.
- [12] Sanmit Narvekar, Bei Peng, Matteo Leonetti, Jivko Sinapov, Matthew E. Taylor, and Peter Stone. Curriculum learning for reinforcement learning domains: A framework and survey. *The Journal of Machine Learning Research (JMLR)*, 21(1):7382–7431, 2020.
- [13] Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, volume 99, pages 278–287, 1999.
- [14] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Scott Ernest, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.
- [15] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [16] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT press, Cambridge, Massachusetts, 2 edition, 2018.
- [17] Mark Towers, J. K. Terry, Ariel Kwiatkowski, John U. Balis, Gianluca Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Arjun KG, Markus Krimmel, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Andrew Tan Jin Shen, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments, 2023. URL <https://github.com/Farama-Foundation/Gymnasium>. Version 0.29.1.

A Anhang